

Análise Exploratória do Comércio Exterior Brasileiro: Foco na Relação com os Estados Unidos

O comércio exterior é um pilar fundamental da economia brasileira, com destaque especial para a relação estratégica com os Estados Unidos. Em face de um cenário global volátil, a perspectiva de novas taxações sobre o intercâmbio comercial entre Brasil e EUA gera apreensão e levanta questões cruciais sobre seus impactos em diversas cadeias produtivas e regiões do país.

Historicamente, os EUA consolidaram-se como um dos principais destinos das exportações brasileiras, absorvendo desde produtos agrícolas e minerais até bens manufaturados de maior valor agregado. Simultaneamente, o Brasil depende das importações americanas de tecnologia, bens de capital e insumos essenciais. Essa interdependência, forjada ao longo de décadas, molda o panorama econômico em níveis nacional, estadual e municipal.

A imposição de novas tarifas ou barreiras comerciais pode desencadear uma série de desafios:

- Aumento dos custos de produção para empresas brasileiras.
- Perda de competitividade dos produtos nacionais no mercado americano.
- Necessidade de reestruturação das cadeias de suprimento.
- Impulso à busca por novos mercados para exportação e importação.

Diante desse quadro, torna-se imperativo quantificar a extensão da dependência comercial do Brasil em relação aos EUA e identificar os riscos potenciais. Tal compreensão é vital para embasar a formulação de políticas públicas resilientes, orientar estratégias empresariais e promover a diversificação de parceiros comerciais.

Esta análise exploratória dedica-se a investigar a relevância dos Estados Unidos e de outros parceiros-chave no comércio exterior brasileiro, buscando oferecer insights sobre a dinâmica atual e os desafios futuros.

✓ Apache Spark

Neste exercício, utilizaremos a plataforma **Google Colab** com **Apache Spark**, uma poderosa ferramenta de processamento de dados distribuído, por meio da sua API em Python chamada **PySpark**. Essa escolha se justifica pela necessidade de manipular dados em escala maior do que a memória de uma única máquina comportaria, com desempenho superior ao de bibliotecas tradicionais como Pandas.

Vantagens do Apache Spark

- **Alta Velocidade:** Processamento em memória (in-memory), muito mais rápido que ferramentas baseadas em disco
- **Escalabilidade Horizontal:** Funciona em clusters, permitindo distribuir a carga entre várias máquinas.
- **API Unificada e Flexível:** Suporta diversas linguagens (Python, Scala, Java, R, SQL) e múltiplos tipos de processamento:
 - **Batch**
 - **Streaming (micro-batching)**
 - **Machine Learning (MLlib)**
 - **Grafos (GraphX)**
- **Tolerância a Falhas:** RDDs permitem recuperação automática em caso de falhas.
- **Sintaxe Familiar (PySpark):** Interface semelhante ao Pandas, facilitando a adoção por quem já trabalha com Python.

Desvantagens do Spark

- **Complexidade de Gerenciamento:** Requer conhecimento sobre clusters e configuração em alguns casos.
- **Custo Elevado:** Especialmente em ambientes de nuvem com grande volume de dados.
- **Streaming com Micro-batches:** Não é ideal para latência em milissegundos.

Considerações Técnicas sobre PySpark

- O Spark é escrito em **Scala** e roda na JVM.
- O **PySpark** atua como uma ponte: comandos Python são traduzidos e executados pelo engine do Spark em Scala/Java.
- Mesmo em ambientes como Google Colab, o Spark opera como um cluster local simplificado.

Conclusão

Ao utilizar o Apache Spark com PySpark, combinamos a **sintaxe acessível do Python** com o **poder da computação distribuída**, ideal para manipulação de dados em larga escala. No entanto, seu uso deve ser cuidadosamente avaliado frente às necessidades e limitações do projeto.

✓ Instalar e importar bibliotecas e funções

```
# Instalar
!pip install --quiet pyspark
```

```
# Importar
import pandas as pd
import numpy as np
import os
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import time
import requests
from datetime import datetime
from IPython.display import clear_output
import gc
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
import pyspark.sql.functions as F
import pyspark.sql.types as T
import subprocess
```

▼ Conectar no google Drive

```
from google.colab import drive
drive.mount('/content/drive/')
```

Mounted at /content/drive/

Fonte dos Dados

Neste exercício, vamos trabalhar com os **dados brutos do comércio exterior brasileiro**, disponibilizados pelo **ComexStat**. Especificamente, utilizaremos os arquivos detalhados por Município da empresa exportadora/importadora e por **Posição do Sistema Harmonizado (SH4)**.

Download e Estrutura dos Dados

Os arquivos estão disponíveis em formato **CSV**, com **separador ponto e vírgula (;)**, e incluem informações por:

- Ano e mês da operação
- Código SH4 do produto
- Código do país de destino (exportação) ou origem (importação)
- Código da UF e do município do domicílio fiscal da empresa
- Quilograma líquido
- Valor FOB em dólares (US\$)

Processamento com Apache Spark

Após o download, os dados serão **carregados e salvos como .parquet**. Isso nos permitirá processar eficientemente grandes volumes de dados de forma distribuída, com melhor desempenho e escalabilidade.

Fonte oficial

Os arquivos podem ser acessados no portal oficial:

[ComexStat - Dados Brutos](#)

▼ Exportações: baixar dados e salvar como `.parquet`

```
# #=====
# # Definir o intervalo de anos que serão baixados os dados
# #=====
# anos = range(2010, 2026)

# #=====
# # Baixar dados de exportação
# #=====
# # Inicializa uma lista vazia para armazenar os DataFrames
# dfs_exp = []

# for ano in anos:
#     # Constrói a URL dinamicamente para cada ano
```

```
# url = f"https://balanca.economia.gov.br/balanca/bd/comexstat-bd/mun/EXP_{ano}_MUN.csv"

# try:
#     # Tenta ler o arquivo CSV da URL
#     print(f"Lendo dados de {ano}...")
#     df = pd.read_csv(url, sep=';')

#     # Adiciona o DataFrame recém-criado à lista 'dfs_exp'
#     dfs_exp.append(df)
#     print(f"-> Sucesso! DataFrame de {ano} adicionado à lista.")

# except Exception as e:
#     # Informa caso o arquivo para um ano específico não seja encontrado ou ocorra outro erro
#     print(f"-> ERRO ao ler o arquivo para o ano {ano}. Motivo: {e}")

# # Unir dataframes
# df_comex_exp = pd.concat(dfs_exp, ignore_index=True)

# #=====
# # Baixar dados e Salvar como .parquet
# #=====
# # Para pandas DataFrame
# df_comex_exp.to_parquet("/content/drive/MyDrive/Projeto_comex_spark/data/dados_comex_mun_export_07-2025.parquet")

# print("Tabela criada e salva")
```

Tabela criada e salva

↯ Importações: baixar dados e salvar como `.parquet`

```
# #=====
# # Definir o intervalo de anos que serão baixados os dados
# #=====
# anos = range(2010, 2026)

# #=====
# # Baixar dados de importações
# #=====
# # Inicializa uma lista vazia para armazenar os DataFrames
# dfs_imp = []

# for ano in anos:
#     # Constrói a URL dinamicamente para cada ano
#     url = f"https://balanca.economia.gov.br/balanca/bd/comexstat-bd/mun/IMP_{ano}_MUN.csv"

#     try:
#         # Tenta ler o arquivo CSV da URL
#         print(f"Lendo dados de {ano}...")
#         df = pd.read_csv(url, sep=';')

#         # Adiciona o DataFrame recém-criado à lista 'dfs_imp'
#         dfs_imp.append(df)
#         print(f"-> Sucesso! DataFrame de {ano} adicionado à lista.")

#     except Exception as e:
#         # Informa caso o arquivo para um ano específico não seja encontrado ou ocorra outro erro
#         print(f"-> ERRO ao ler o arquivo para o ano {ano}. Motivo: {e}")

# # Unir dataframes
# df_comex_imp = pd.concat(dfs_imp, ignore_index=True)

# #=====
# # Baixar dados e Salvar como .parquet
# #=====
# # Para pandas DataFrame
# df_comex_imp.to_parquet("/content/drive/MyDrive/Projeto_comex_spark/data/dados_comex_mun_import_07-2025.parquet")

# print("Tabela criada e salva")
```

Dicionários e Classificações da Balança Comercial – COMEXSTAT

Este documento descreve os principais dicionários usados nas análises da balança comercial brasileira e como integrá-los usando a chave `CO_NCM`.

Descrição dos Dicionários

NCM.csv – Nomenclatura Comum do Mercosul

- Código de 8 dígitos que identifica mercadorias.
- Colunas principais:
 - **CO_NCM**: código NCM
 - **NO_NCM_POR**: descrição do produto
 - **CO_FAT_AGREG**: fator agregado (produto primário, consumo, etc.)

NCM_CGCE.csv – Classificação por Grandes Categorias Econômicas

- Agrupa os produtos em categorias macroeconômicas como bens de consumo e bens de capital.
- Colunas:
 - **CO_CGCE_N1**, **CO_CGCE_N2**, **CO_CGCE_N3**
 - **NO_CGCE_N3**: descrição da categoria
 - **CO_NCM**: chave para unir

NCM_PPE.csv – Pauta de Produtos Exportados

- Classificação temática para exportações.
- Colunas:
 - **CO_PPE**, **NO_PPE**
 - **CO_NCM**: chave para unir

NCM_PPI.csv – Pauta de Produtos Importados

- Versão da PPE para importações.
- Colunas:
 - **CO_PPI**, **NO_PPI**
 - **CO_NCM**: chave para unir

NCM_UNIDADE.csv – Unidades Estatísticas

- Define a unidade de medida usada para cada NCM.
- Colunas:
 - **CO_UNID**, **NO_UNID**
 - **CO_NCM**: chave para unir

CLASSIFICACAO-SEMANAL-ANTIGA_x_ISIC_x_CUCI-GRUPO.xlsx

- Conversão entre classificações semanais antigas e sistemas internacionais ISIC e CUCI.
- Contém colunas como:
 - **SECAO_ISIC**, **GRUPO_CUCI**, **NO_CUCI**

Esquema de JOINS entre os Dicionários COMEXSTAT

Abaixo, mostro como organizar os joins com base em suas chaves comuns. Após isso, apresento o código em Python para unir os arquivos de forma prática.

1. Dicionários baseados em NCM

Esses arquivos podem ser unidos usando **CO_NCM**:

- **NCM.csv** → Base principal, com a descrição do produto
- **NCM_SH.csv** → Liga NCM ao código SH
- **NCM_CUCI.csv** → Liga NCM à classificação CUCI
- **NCM_CGCE.csv** → Liga NCM à Classificação por Grandes Categorias Econômicas
- **NCM_PPE.csv** → Liga NCM à Pauta de Produtos Exportados
- **NCM_PPI.csv** → Liga NCM à Pauta de Produtos Importados
- **NCM_UNIDADE.csv** → Liga NCM à unidade estatística
- **NBM.csv** → Nomenclatura anterior (1989–1996)

▼ **Baixar dicionários e planilhas auxiliares**

```

# # =====
# # Baixar tabelas e gerar dataframes
# # =====
# urls = {
#     "pais": "https://balanca.economia.gov.br/balanca/bd/tabelas/PAIS.csv",
#     "bloco": "https://balanca.economia.gov.br/balanca/bd/tabelas/PAIS_BLOCO.csv",
#     "uf_mun": "https://balanca.economia.gov.br/balanca/bd/tabelas/UF_MUN.csv",
#     "uf": "https://balanca.economia.gov.br/balanca/bd/tabelas/UF.csv",
#     "via": "https://balanca.economia.gov.br/balanca/bd/tabelas/VIA.csv",
#     "urf": "https://balanca.economia.gov.br/balanca/bd/tabelas/URF.csv",
#     "ncm": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM.csv",
#     "ncm_sh": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM_SH.csv",
#     "cuci": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM_CUCI.csv",
#     "isic_cuci": "https://balanca.economia.gov.br/balanca/bd/tabelas/ISIC_CUCI.csv",
#     "cgce": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM_CGCE.csv",
#     "fator_a": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM_FAT_AGREG.csv",
#     "ppe": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM_PPE.csv",
#     "ppi": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM_PPI.csv",
#     "unid_estat": "https://balanca.economia.gov.br/balanca/bd/tabelas/NCM_UNIDADE.csv",
#     "nbm_1996": "https://balanca.economia.gov.br/balanca/bd/tabelas/NBM.csv",
#     "nbm_x_ncm": "https://balanca.economia.gov.br/balanca/bd/tabelas/NBM_NCM.csv",
# }

# # Gerar dataframes automaticamente
# for nome, url in urls.items():
#     globals()[f'df_{nome}'] = pd.read_csv(url, sep=';', encoding='latin-1')
#     print(f'df_{nome}: {globals()[f'df_{nome}'].shape}')

# print("Geração dos dataframes concluída")

# # =====
# # UNIÃO POR GRUPOS TEMÁTICOS
# # =====

# # Geografia completa
# df_geo_master = df_uf_mun.merge(df_uf, on='SG_UF', how='left')

# # _____

# # Países + blocos econômicos
# df_paises_master = df_pais.merge(df_bloco, on='CO_PAIS', how='left')

# # selecionar colunas uteis
# df_paises_master_select = df_paises_master[['CO_PAIS', 'CO_PAIS_ISON3', 'CO_PAIS_ISO3', 'NO_PAIS', 'NO_BLOCO']]

# # _____

# # NCM com todas as classificações (dataset principal)
# df_produtos_master = (df_ncm
#     .merge(df_ncm_sh, on='CO_SH6', how='left')
#     .merge(df_cuci, on='CO_CUCI_ITEM', how='left')
#     .merge(df_cgce, on='CO_CGCE_N3', how='left')
#     .merge(df_fator_a, on='CO_FAT_AGREG', how='left')
#     .merge(df_ppe, on='CO_PPE', how='left')
#     .merge(df_ppi, on='CO_PPI', how='left')
#     .merge(df_unid_estat, on='CO_UNID', how='left')
# )

# # selecionar colunas uteis
# df_produtos_master_select = df_produtos_master[['CO_SH4', 'NO_NCM_POR', 'NO_CUCI_GRUPO', 'NO_CUCI_DIVISAO', 'NO_PPE_MIN',
#     'NO_CGCE_N2', 'NO_CGCE_N1', 'NO_FAT_AGREG', 'CO_CUCI_GRUPO']]

# # _____

# # CUCI + ISIC
# df_cuci_master = df_cuci.merge(df_isic_cuci, on='CO_CUCI_GRUPO', how='left')

# # visualizar
# # Select only the necessary columns and drop the duplicate one before merging
# df_cuci_master_select = df_cuci_master[['NO_ISIC_SECAO', 'CO_CUCI_GRUPO']].drop_duplicates(subset=['CO_CUCI_GRUPO'])

# # _____

# # df_produtos_master_select + df_cuci_master_select econômicos
# df_produtoscuci_master_select = df_produtos_master_select.merge(df_cuci_master_select, on='CO_CUCI_GRUPO', how='left')

# print("Combinação dos dicionários concluída")

# # =====
# # SALVAR
# # =====

# # Lista completa com todos os DataFrames
# dataframes = {
#     # Novos dicionários das URLs
#     "pais": df_pais,

```

```

# "bloco": df_bloco,
# "uf_mun": df_uf_mun,
# "uf": df_uf,
# "via": df_via,
# "urf": df_urf,
# "ncm": df_ncm,
# "ncm_sh": df_ncm_sh,
# "cuci": df_cuci,
# "isic_cuci": df_isic_cuci,
# "cgce": df_cgce,
# "fator_a": df_fator_a,
# "ppe": df_ppe,
# "ppi": df_ppi,
# "unid_estat": df_unid_estat,
# "nbm_1996": df_nbm_1996,
# "nbm_x_ncm": df_nbm_x_ncm,

# # Dicionários modificados
# "df_geo_master" : df_geo_master,
# "df_paises_master" : df_paises_master,
# "df_paises_master_select" : df_paises_master_select,
# "df_produtos_master" : df_produtos_master,
# "df_cuci_master" : df_cuci_master,
# "df_cuci_master_select" : df_cuci_master_select,
# "df_produtoscuci_master_select" : df_produtoscuci_master_select,
# }

# # Caminho base
# base_path = "/content/drive/MyDrive/Projeto_comex_spark/data/"

# # Criar diretório se não existir
# os.makedirs(base_path, exist_ok=True)

# # Salvar todos os DataFrames
# print("Salvando DataFrames...")
# for nome, df in dataframes.items():
#     try:
#         caminho_completo = f"{base_path}{nome}.parquet"
#         df.to_parquet(caminho_completo, index=False)
#         print(f"Salvo: {nome}.parquet")
#     except Exception as e:
#         print(f"Erro ao salvar {nome}: {e}")

# print(f"\n Processo concluído! Total de {len(dataframes)} DataFrames processados.")

```

▼ Definir import dos dados

```

# Criar/reactivar sessão Spark
spark = SparkSession.builder \
    .appName("COMEX Analysis") \
    .getOrCreate()

# Caminho base dos arquivos parquet
base_path = "/content/drive/MyDrive/data/Comercio_exterior_spark_análise/"

# Arquivos prontos para consultas
df_export = spark.read.parquet(f"{base_path}dados_comex_mun_export_07-2025.parquet")
df_import = spark.read.parquet(f"{base_path}dados_comex_mun_import_07-2025.parquet")

# Verificar estrutura (lazy evaluation - não carrega tudo)
print(f"\nEstrutura dos dados:")
print(f"Exportação: {df_export.count()} linhas")
print(f"Importação: {df_import.count()} linhas")

# Ver schema sem carregar dados
print("\nSchemas:")
df_export.printSchema()
df_import.printSchema()

```

Estrutura dos dados:
 Exportação: 13779191 linhas
 Importação: 22702125 linhas

Schemas:
 root
 |-- CO_ANO: long (nullable = true)
 |-- CO_MES: long (nullable = true)
 |-- SH4: long (nullable = true)

```

|-- CO_PAIS: long (nullable = true)
|-- SG_UF_MUN: string (nullable = true)
|-- CO_MUN: long (nullable = true)
|-- KG_LIQUIDO: long (nullable = true)
|-- VL_FOB: long (nullable = true)

root
 |-- CO_ANO: long (nullable = true)
 |-- CO_MES: long (nullable = true)
 |-- SH4: long (nullable = true)
 |-- CO_PAIS: long (nullable = true)
 |-- SG_UF_MUN: string (nullable = true)
 |-- CO_MUN: long (nullable = true)
 |-- KG_LIQUIDO: long (nullable = true)
 |-- VL_FOB: long (nullable = true)

```

✓ Análise Exploratória Simples

✓ Período de dados disponíveis

```

inicio = time.time()

# EXPORTAÇÃO - Select apenas das colunas necessárias + criação da data temporária
print("=== EXPORTAÇÃO ===")
df_export.select("CO_ANO", "CO_MES") \
    .withColumn(
        "DATA_TEMP",
        to_date(concat(
            col("CO_ANO").cast("string"),
            lit("-"),
            lpad(col("CO_MES").cast("string"), 2, "0"),
            lit("-01")
        ), "yyyy-MM-dd")
    ) \
    .select(
        min("DATA_TEMP").alias("data_minima"),
        max("DATA_TEMP").alias("data_maxima")
    ).show()

# IMPORTAÇÃO - Select apenas das colunas necessárias + criação da data temporária
print("=== IMPORTAÇÃO ===")
df_import.select("CO_ANO", "CO_MES") \
    .withColumn(
        "DATA_TEMP",
        to_date(concat(
            col("CO_ANO").cast("string"),
            lit("-"),
            lpad(col("CO_MES").cast("string"), 2, "0"),
            lit("-01")
        ), "yyyy-MM-dd")
    ) \
    .select(
        min("DATA_TEMP").alias("data_minima"),
        max("DATA_TEMP").alias("data_maxima")
    ).show()

fim = time.time()
print(f"Tempo total: {fim - inicio:.3f} segundos")

```

```

=== EXPORTAÇÃO ===
+-----+-----+
|data_minima|data_maxima|
+-----+-----+
| 2010-01-01| 2025-06-01|
+-----+-----+

```

```

=== IMPORTAÇÃO ===
+-----+-----+
|data_minima|data_maxima|
+-----+-----+
| 2010-01-01| 2025-06-01|
+-----+-----+

```

Tempo total: 53.282 segundos

✓ Análises de missing values

```
# Análise de Valores Ausentes (df Exportações)
print("\n--- Contagem de Valores Ausentes por Coluna ---")
for col_name in df_export.columns:
    null_count = df_export.filter(col(col_name).isNull()).count()
    print(f"Coluna '{col_name}': {null_count} valores ausentes")

# Análise de Valores Ausentes (df Importações)
print("\n--- Contagem de Valores Ausentes por Coluna ---")
for col_name in df_import.columns:
    null_count = df_import.filter(col(col_name).isNull()).count()
    print(f"Coluna '{col_name}': {null_count} valores ausentes")
```

```
--- Contagem de Valores Ausentes por Coluna ---
Coluna 'CO_ANO': 0 valores ausentes
Coluna 'CO_MES': 0 valores ausentes
Coluna 'SH4': 0 valores ausentes
Coluna 'CO_PAIS': 0 valores ausentes
Coluna 'SG_UF_MUN': 0 valores ausentes
Coluna 'CO_MUN': 0 valores ausentes
Coluna 'KG_LIQUIDO': 0 valores ausentes
Coluna 'VL_FOB': 0 valores ausentes
```

```
--- Contagem de Valores Ausentes por Coluna ---
Coluna 'CO_ANO': 0 valores ausentes
Coluna 'CO_MES': 0 valores ausentes
Coluna 'SH4': 0 valores ausentes
Coluna 'CO_PAIS': 0 valores ausentes
Coluna 'SG_UF_MUN': 0 valores ausentes
Coluna 'CO_MUN': 0 valores ausentes
Coluna 'KG_LIQUIDO': 0 valores ausentes
Coluna 'VL_FOB': 0 valores ausentes
```

▼ Estatísticas Descritivas

```
# Análise Descritiva para Colunas Numéricas - EXPORTAÇÃO
print("\nEstatísticas Descritivas - EXPORTAÇÃO:")
df_export_stats = df_export.describe('KG_LIQUIDO', 'VL_FOB')

# Arredondar valores numéricos
df_export_stats_rounded = df_export_stats.select(
    "summary",
    round(col("KG_LIQUIDO"), 0).alias("peso_liquido_total_kg"),
    round(col("VL_FOB"), 0).alias("valor_fob_dolar_total")
)

df_export_stats_pd = df_export_stats_rounded.toPandas().T
print(df_export_stats_pd)

# Análise Descritiva para Colunas Numéricas - IMPORTAÇÃO
print("\nEstatísticas Descritivas - IMPORTAÇÃO:")
df_import_stats = df_import.describe('KG_LIQUIDO', 'VL_FOB')

# Arredondar valores numéricos
df_import_stats_rounded = df_import_stats.select(
    "summary",
    round(col("KG_LIQUIDO"), 2).alias("peso_liquido_total_kg"),
    round(col("VL_FOB"), 2).alias("valor_fob_dolar_total")
)

df_import_stats_pd = df_import_stats_rounded.toPandas().T
print(df_import_stats_pd)

# Limpeza de memória
del df_export_stats, df_export_stats_rounded, df_export_stats_pd
del df_import_stats, df_import_stats_rounded, df_import_stats_pd
```

Estatísticas Descritivas - EXPORTAÇÃO:

	0	1	2	3	4
summary	count	mean	stddev	min	max
peso_liquido_total_kg	13779191.0	743660.0	30904457.0	0.0	24030871000.0
valor_fob_dolar_total	13779191.0	279303.0	4419091.0	0.0	1733895757.0

Estatísticas Descritivas - IMPORTAÇÃO:

	0	1	2	3	4
summary	count	mean	stddev	min	max
peso_liquido_total_kg	22702125.0	106997.89	3208717.08	0.0	1823339652.0
valor_fob_dolar_total	22702125.0	142791.45	2393657.24	0.0	2661051787.0

Análise exploratória Econômica e Comercial

Série Histórica Anual das Exportações e Importações Brasileiras

```

=====
# Análise de Exportações e Importações por Ano
=====

print("Análise comparativa: Exportações vs Importações (2010 - Jun/2025)")

# Dados de Exportação
exportacoes_por_ano = df_export.groupby("CO_ANO").agg(
    round(sum("VL_FOB"), 2).alias("valor_fob_export"),
    round(sum("KG_LIQUIDO"), 2).alias("peso_export")
).orderBy("CO_ANO")

# Dados de Importação
importacoes_por_ano = df_import.groupby("CO_ANO").agg(
    round(sum("VL_FOB"), 2).alias("valor_fob_import"),
    round(sum("KG_LIQUIDO"), 2).alias("peso_import")
).orderBy("CO_ANO")

# Converter para pandas
exp_ano_pd = exportacoes_por_ano.toPandas()
imp_ano_pd = importacoes_por_ano.toPandas()

# Merge dos dados por ano
dados_completos = exp_ano_pd.merge(imp_ano_pd, on='CO_ANO', how='outer').fillna(0)

=====
# Dashboard Plotly Vertical (2x1) - Exportações vs Importações
=====

# Criar subplots verticais (2 linhas, 1 coluna)
fig = make_subplots(
    rows=2, cols=1,
    subplot_titles=(
        '<b>Valor FOB Total - Exportações vs Importações</b>',
        '<b>Peso Líquido Total - Exportações vs Importações</b>'
    ),
    vertical_spacing=0.20 # Mais espaço para acomodar legenda embaixo
)

# Função para formatar números
def formatar_valor(valor):
    if valor >= 1e9:
        return f"${valor/1e9:.1f}B"
    elif valor >= 1e6:
        return f"${valor/1e6:.1f}M"
    else:
        return f"${valor:,.0f}"

def formatar_peso(peso):
    if peso >= 1e9:
        return f"{peso/1e9:.1f}B kg"
    elif peso >= 1e6:
        return f"{peso/1e6:.1f}M kg"
    else:
        return f"{peso:,.0f} kg"

# 1. GRÁFICO SUPERIOR: Valores FOB
# Exportações
fig.add_trace(
    go.Scatter(
        x=dados_completos['CO_ANO'],
        y=dados_completos['valor_fob_export'],
        mode='lines+markers',
        name='Exportações',
        line=dict(color='darkgreen', width=3),
        marker=dict(size=8, color='darkgreen'),
        hovertemplate='<b>{x}</b><br><b>Exportação:</b> {text}<extra></extra>',
        text=[formatar_valor(v) for v in dados_completos['valor_fob_export']]
    ),
    row=1, col=1
)

# Importações
fig.add_trace(
    go.Scatter(

```

```

x=dados_completos['CO_ANO'],
y=dados_completos['valor_fob_import'],
mode='lines+markers',
name='Importações',
line=dict(color='steelblue', width=3),
marker=dict(size=8, color='steelblue', symbol='square'),
hovertemplate='<b>{x}</b><br><b>Importação:</b> {text}<extra></extra>',
text=[formatar_valor(v) for v in dados_completos['valor_fob_import']]
),
row=1, col=1
)

# 2. GRÁFICO INFERIOR: Peso Líquido
# Exportações
fig.add_trace(
    go.Scatter(
        x=dados_completos['CO_ANO'],
        y=dados_completos['peso_export'],
        mode='lines+markers',
        name='Exportações (Peso)',
        line=dict(color='darkgreen', width=3),
        marker=dict(size=8, color='darkgreen'),
        hovertemplate='<b>{x}</b><br><b>Exportação:</b> {text}<extra></extra>',
        text=[formatar_peso(p) for p in dados_completos['peso_export']],
        showlegend=False # Não mostrar na legenda (já está no gráfico superior)
    ),
    row=2, col=1
)

# Importações
fig.add_trace(
    go.Scatter(
        x=dados_completos['CO_ANO'],
        y=dados_completos['peso_import'],
        mode='lines+markers',
        name='Importações (Peso)',
        line=dict(color='steelblue', width=3),
        marker=dict(size=8, color='steelblue', symbol='square'),
        hovertemplate='<b>{x}</b><br><b>Importação:</b> {text}<extra></extra>',
        text=[formatar_peso(p) for p in dados_completos['peso_import']],
        showlegend=False # Não mostrar na legenda (já está no gráfico superior)
    ),
    row=2, col=1
)

# Configurar layout com legenda na parte inferior
fig.update_layout(
    title={
        'text': '<b>Comércio Exterior Brasileiro</b> - Exportações vs Importações (2010 - Jun/2025)',
        'x': 0.5,
        'font': {'size': 20, 'family': 'Arial Black'}
    },
    height=900, # Aumentar altura para acomodar legenda embaixo
    width=1200,
    font=dict(family="Arial", size=12),
    plot_bgcolor='white',
    paper_bgcolor='white',
    legend=dict(
        orientation="h", # Horizontal
        yanchor="top", # Ancorar no topo da legenda
        y=-0.12, # Posição abaixo do gráfico (valor negativo)
        xanchor="center", # Centralizar horizontalmente
        x=0.5, # Centro do gráfico
        bgcolor="rgba(255,255,255,0.9)", # Fundo branco
        bordercolor="lightgray",
        borderwidth=1,
        font=dict(size=12)
    )
)

# Configurar eixos
fig.update_xaxes(title_text="<b>Ano</b>", showgrid=True, gridcolor='#E5E5E5')
fig.update_yaxes(title_text="<b>Valor FOB (USD)</b>", showgrid=True, gridcolor='#E5E5E5', row=1, col=1)
fig.update_yaxes(title_text="<b>Peso Líquido (kg)</b>", showgrid=True, gridcolor='#E5E5E5', row=2, col=1)

# Mostrar gráfico
fig.show()

# LIMPEZA DE MEMÓRIA
del exportacoes_por_ano, importacoes_por_ano, exp_ano_pd, imp_ano_pd, dados_completos

```

Análise comparativa: Exportações vs Importações (2010 - Jun/2025)

Comércio Exterior Brasileiro - Exportações vs Importações (2010 - Jun/2025)✓ **Análise Resumida — Comércio Exterior Brasileiro (2010–jun/2025)****Tendência Geral**

- Exportações > importações (superávit).
- Maior volatilidade a partir de 2020.

Valor FOB (USD)

- Alta até 2013; queda 2014–2016; recuperação 2017–2019.
- Pico em 2022–2023 (> US\$ 300 bi).
- Queda acentuada em 2025 (preços/demanda).

Peso Líquido (kg)

- Exportações crescentes até 2023; leve queda em 2025.
- Importações estáveis e muito menores.
- Indica exportação de bens pesados/baixo valor unitário e importação de bens leves/alto valor.

Implicações

- Superávit dependente de commodities.
- Sensível a choques externos e tarifas.
- Necessidade de diversificação de mercados/produtos.

✓ Série Histórica Mensal das Exportações e Importações Brasileiras

```

=====
# Análise de Exportações E Importações por Mês
=====

print("Análise comparativa mensal: Exportações vs Importações (Jan/2010 - Jun/2025)")

# Dados de Exportação por mês
exportacoes_por_mes = df_export.groupby("CO_ANO", "CO_MES").agg(
    round(sum("VL_FOB"), 2).alias("valor_fob_export"),
    round(sum("KG_LIQUIDO"), 2).alias("peso_export")
).orderBy("CO_ANO", "CO_MES")

# Dados de Importação por mês
importacoes_por_mes = df_import.groupby("CO_ANO", "CO_MES").agg(
    round(sum("VL_FOB"), 2).alias("valor_fob_import"),
    round(sum("KG_LIQUIDO"), 2).alias("peso_import")
).orderBy("CO_ANO", "CO_MES")

# Converter para pandas
exp_mes_pd = exportacoes_por_mes.toPandas()
imp_mes_pd = importacoes_por_mes.toPandas()

# Criar coluna de data para melhor visualização
exp_mes_pd['data'] = exp_mes_pd['CO_ANO'].astype(str) + '-' + exp_mes_pd['CO_MES'].astype(str).str.zfill(2)
imp_mes_pd['data'] = imp_mes_pd['CO_ANO'].astype(str) + '-' + imp_mes_pd['CO_MES'].astype(str).str.zfill(2)

# Converter para datetime
exp_mes_pd['data'] = pd.to_datetime(exp_mes_pd['data'], format='%Y-%m')
imp_mes_pd['data'] = pd.to_datetime(imp_mes_pd['data'], format='%Y-%m')

# Merge dos dados por mês
dados_completos = exp_mes_pd.merge(imp_mes_pd, on=['CO_ANO', 'CO_MES', 'data'], how='outer').fillna(0)
dados_completos = dados_completos.sort_values('data')

print(f"Período: {dados_completos['data'].min()} até {dados_completos['data'].max()}")

=====
# Dashboard Plotly Vertical (2x1) - Série Temporal Mensal
=====

# Criar subplots verticais (2 linhas, 1 coluna)
fig = make_subplots(
    rows=2, cols=1,
    subplot_titles=(
        '<b>Valor FOB Total Mensal - Exportações vs Importações</b>',
        '<b>Peso Líquido Total Mensal - Exportações vs Importações</b>'
    ),
    vertical_spacing=0.20
)

# Função para formatar números
def formatar_valor(valor):
    if valor >= 1e9:
        return f"${valor/1e9:.1f}B"
    elif valor >= 1e6:
        return f"${valor/1e6:.1f}M"
    else:
        return f"${valor:,.0f}"

def formatar_peso(peso):
    if peso >= 1e9:
        return f"{peso/1e9:.1f}B kg"
    elif peso >= 1e6:
        return f"{peso/1e6:.1f}M kg"
    else:
        return f"{peso:,.0f} kg"

# 1. GRÁFICO SUPERIOR: Valores FOB
# Exportações
fig.add_trace(
    go.Scatter(
        x=dados_completos['data'],
        y=dados_completos['valor_fob_export'],
        mode='lines', # Sem markers para dados mensais (muitos pontos)
        name='Exportações',
        line=dict(color='darkgreen', width=2),
        hovertemplate='<b>%{x}%Y-%m</b><br><b>Exportação:</b> <extra> %<text><extra>',
        text=[formatar_valor(v) for v in dados_completos['valor_fob_export']]
    ),

```

```

    row=1, col=1
)

# Importações
fig.add_trace(
    go.Scatter(
        x=dados_completos['data'],
        y=dados_completos['valor_fob_import'],
        mode='lines',
        name='Importações',
        line=dict(color='steelblue', width=2),
        hovertemplate='<b>%{x|%Y-%m}</b><br><b>Importação:</b> %{text}<extra></extra>',
        text=[formatar_valor(v) for v in dados_completos['valor_fob_import']]
    ),
    row=1, col=1
)

# 2. GRÁFICO INFERIOR: Peso Líquido
# Exportações
fig.add_trace(
    go.Scatter(
        x=dados_completos['data'],
        y=dados_completos['peso_export'],
        mode='lines',
        name='Exportações (Peso)',
        line=dict(color='darkgreen', width=2),
        hovertemplate='<b>%{x|%Y-%m}</b><br><b>Exportação:</b> %{text}<extra></extra>',
        text=[formatar_peso(p) for p in dados_completos['peso_export']],
        showlegend=False
    ),
    row=2, col=1
)

# Importações
fig.add_trace(
    go.Scatter(
        x=dados_completos['data'],
        y=dados_completos['peso_import'],
        mode='lines',
        name='Importações (Peso)',
        line=dict(color='steelblue', width=2),
        hovertemplate='<b>%{x|%Y-%m}</b><br><b>Importação:</b> %{text}<extra></extra>',
        text=[formatar_peso(p) for p in dados_completos['peso_import']],
        showlegend=False
    ),
    row=2, col=1
)

# Configurar layout
fig.update_layout(
    title={
        'text': '<b>Comércio Exterior Brasileiro</b> - Série Temporal Mensal (Jan/2010 - Jun/2025)',
        'x': 0.5,
        'font': {'size': 20, 'family': 'Arial Black'}
    },
    height=900,
    width=1200, # Mais largo para dados mensais
    font=dict(family="Arial", size=12),
    plot_bgcolor='white',
    paper_bgcolor='white',
    legend=dict(
        orientation="h",
        yanchor="top",
        y=-0.12,
        xanchor="center",
        x=0.5,
        bgcolor="rgba(255,255,255,0.9)",
        bordercolor="lightgray",
        borderwidth=1,
        font=dict(size=12)
    )
)

# Configurar eixos
fig.update_xaxes(
    title_text="<b>Período</b>",
    showgrid=True,
    gridcolor='#E5E5E5',
    tickformat='%Y-%m' # Formato de data
)

fig.update_yaxes(title_text="<b>Valor FOB (USD)</b>", showgrid=True, gridcolor='#E5E5E5', row=1, col=1)
fig.update_yaxes(title_text="<b>Peso Líquido (kg)</b>", showgrid=True, gridcolor='#E5E5E5', row=2, col=1)

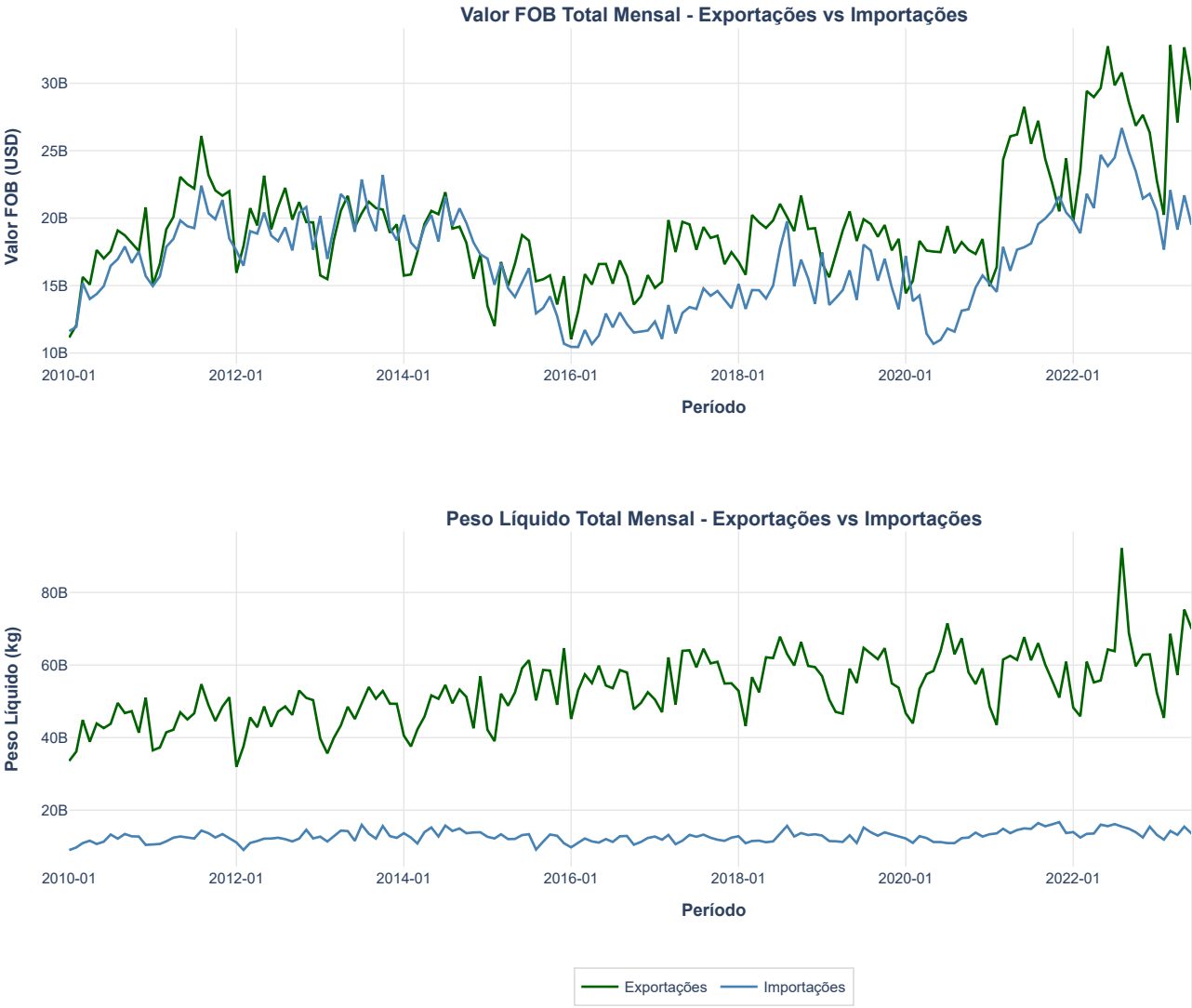
```

```
# Mostrar gráfico
fig.show()

# LIMPEZA DE MEMÓRIA
del exportacoes_por_mes, importacoes_por_mes, exp_mes_pd, imp_mes_pd, dados_completos
```

Análise comparativa mensal: Exportações vs Importações (Jan/2010 - Jun/2025)
Período: 2010-01-01 00:00:00 até 2025-06-01 00:00:00

Comércio Exterior Brasileiro - Série Temporal Mensal (Jan/2010 - Jun/2025)



✦ Análise Geral — Comércio Exterior Brasileiro (Jan/2010 – Jun/2025)

Valor FOB (USD)

- Exportações e importações exibem **tendência de crescimento no longo prazo**, com variações cíclicas.
- **2010–2014**: aumento consistente do comércio; Brasil beneficiado por preços altos de commodities.
- **2015–2016**: queda acentuada ligada à recessão e à queda global de preços.
- **2017–2020**: recuperação gradual, seguida por leve desaceleração no início da pandemia.
- **2021–2023**: forte expansão, impulsionada por commodities energéticas e agrícolas.
- **2024–2025**: estabilização em patamar alto, mas com **volatilidade mensal acentuada**.

Peso Líquido (kg)

- A série mostra **crescimento moderado e contínuo** das exportações físicas.
- Importações mantêm volume bem inferior e estável.
- Diferença entre valor e peso indica **maior valor agregado das importações** frente às exportações baseadas em volume (commodities).

Sazonalidade

- Exportações têm **padrão sazonal claro**: picos entre **março e agosto**, coincidindo com períodos de safra e embarques agrícolas.
- Importações apresentam **picos mais suaves e distribuição uniforme** ao longo do ano, refletindo compras contínuas de bens industriais e tecnológicos.
- O comportamento sazonal se manteve consistente ao longo da série, embora **a amplitude das oscilações tenha aumentado** após 2020.

Interpretação Econômica

- A economia brasileira mantém **superávit estrutural**, mas fortemente dependente de produtos primários.
- A volatilidade pós-2020 mostra **maior sensibilidade a choques externos** (pandemia, guerra, variação de preços de energia).
- A sazonalidade reforça a **dependência de ciclos agrícolas e de commodities**, evidenciando a importância de políticas de diversificação exportadora.

```
#=====
# Análise de Exportações E Importações por Mês
#=====

print("Análise comparativa mensal: Exportações vs Importações (Jan/2010 - Jun/2025)")

# Dados de Exportação por mês
exportacoes_por_mes = df_export.groupby("CO_ANO", "CO_MES").agg(
    round(sum("VL_FOB"), 2).alias("valor_fob_export"),
    round(sum("KG_LIQUIDO"), 2).alias("peso_export")
).orderBy("CO_ANO", "CO_MES")

# Dados de Importação por mês
importacoes_por_mes = df_import.groupby("CO_ANO", "CO_MES").agg(
    round(sum("VL_FOB"), 2).alias("valor_fob_import"),
    round(sum("KG_LIQUIDO"), 2).alias("peso_import")
).orderBy("CO_ANO", "CO_MES")

# Converter para pandas
exp_mes_pd = exportacoes_por_mes.toPandas()
imp_mes_pd = importacoes_por_mes.toPandas()

# Criar coluna de data para melhor visualização
exp_mes_pd['data'] = exp_mes_pd['CO_ANO'].astype(str) + '-' + exp_mes_pd['CO_MES'].astype(str).str.zfill(2)
imp_mes_pd['data'] = imp_mes_pd['CO_ANO'].astype(str) + '-' + imp_mes_pd['CO_MES'].astype(str).str.zfill(2)

# Converter para datetime
exp_mes_pd['data'] = pd.to_datetime(exp_mes_pd['data'], format='%Y-%m')
imp_mes_pd['data'] = pd.to_datetime(imp_mes_pd['data'], format='%Y-%m')

# Merge dos dados por mês
dados_completos = exp_mes_pd.merge(imp_mes_pd, on=['CO_ANO', 'CO_MES', 'data'], how='outer').fillna(0)
dados_completos = dados_completos.sort_values('data')

#=====
# Plot Plotly
#=====

# Calcular saldo comercial
dados_completos['saldo_comercial'] = dados_completos['valor_fob_export'] - dados_completos['valor_fob_import']

# Separar dados de superávit e déficit
superavit = dados_completos[dados_completos['saldo_comercial'] >= 0]
deficit = dados_completos[dados_completos['saldo_comercial'] < 0]

# Criar figura
fig = go.Figure()

# Superávit
fig.add_trace(
    go.Bar(
        x=superavit['data'],
        y=superavit['saldo_comercial'],
        name='Superávit',
        marker_color='green',
        hovertemplate='<b>{x|Y-%m}</b><br><b>Saldo:</b> {y:,.0f} USD<extra></extra>'
    )
)

# Déficit
fig.add_trace(
    go.Bar(
        x=deficit['data'],
        y=deficit['saldo_comercial'],
        name='Déficit',
```

```

marker_color='red',
hovertemplate='<b>{x|%Y-%m}</b><br><b>Saldo:</b> {y:,.0f} USD<extra></extra>'
)
)

# Layout e legenda
fig.update_layout(
    title='<b>Saldo da Balança Comercial Brasileira (Mensal)</b><br>(Exportações - Importações)',
    xaxis_title='Período',
    yaxis_title='Saldo (USD)',
    plot_bgcolor='white',
    paper_bgcolor='white',
    height=500,
    width=1000,
    font=dict(family="Arial", size=12),
    legend=dict(
        orientation="h",
        yanchor="bottom",
        y=-0.25,
        xanchor="center",
        x=0.5,
        bgcolor="rgba(255,255,255,0.9)",
        bordercolor="lightgray",
        borderwidth=1
    )
)

fig.update_xaxes(showgrid=True, gridcolor='lightgray', tickformat='%Y-%m')
fig.update_yaxes(showgrid=True, gridcolor='lightgray')

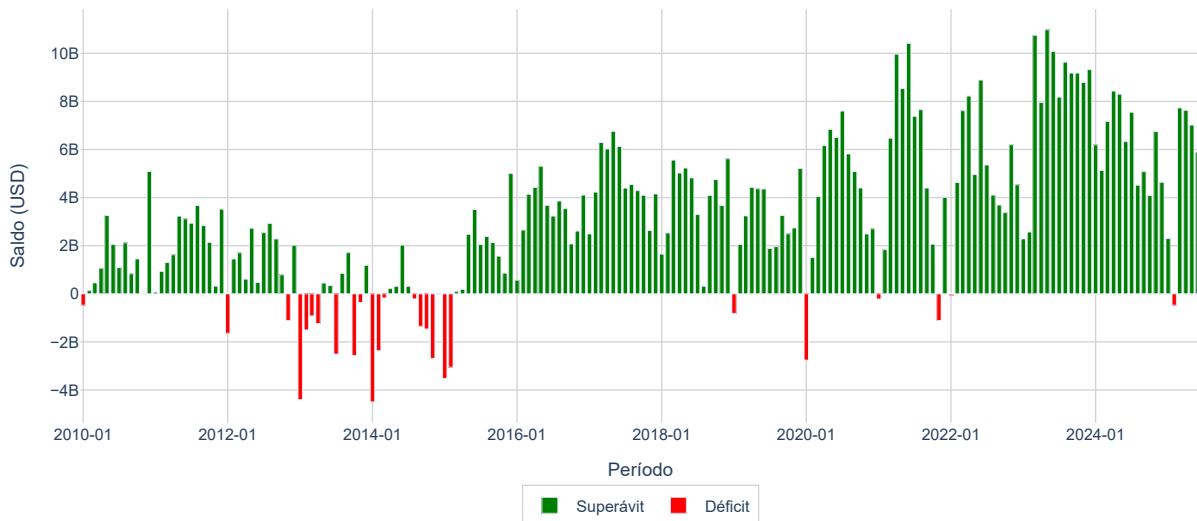
fig.show()

# LIMPEZA DE MEMÓRIA
del exportacoes_por_mes, importacoes_por_mes, exp_mes_pd, imp_mes_pd, dados_completos

```

Análise comparativa mensal: Exportações vs Importações (Jan/2010 - Jun/2025)

Saldo da Balança Comercial Brasileira (Mensal) (Exportações - Importações)



✓ Análise do Saldo da Balança Comercial Brasileira (Jan/2010 – Jun/2025)

Panorama Geral

- O Brasil apresenta **predomínio de superávits mensais** ao longo de todo o período.
- Os saldos positivos (barras verdes) tornam-se mais expressivos após **2016**, consolidando um ciclo de melhora na balança comercial.
- Os déficits (barras vermelhas) concentram-se entre **2012 e 2015**, período de forte desaceleração econômica e queda no preço das commodities.

Tendências e Ciclos

- **2010–2014**: alternância entre superávits e déficits, refletindo vulnerabilidade externa e alta dependência de importações industriais.
- **2015–2016**: inflexão positiva, com melhora das exportações e retração das importações.
- **2017–2020**: saldo consistente e crescente, mesmo com choques pontuais (pandemia).

- **2021–2023:** pico de superávit mensal superior a **US\$ 10 bilhões**, impulsionado pelo boom de commodities.
- **2024–2025:** leve redução do saldo, mas ainda mantendo superávits.

Interpretação Econômica

- O **superávit estrutural** indica que o Brasil exporta mais do que importa de forma recorrente.
- A elevação após 2016 reflete:
 - Queda nas importações de bens de capital e intermediários durante a recessão.
 - Aumento da demanda global por produtos primários (soja, petróleo, minério).
- A partir de 2020, a **volatilidade dos saldos mensais** aumenta, sugerindo sensibilidade a preços internacionais e variação cambial.

Conclusão

- O desempenho comercial reforça a **dependência de commodities** e a **baixa diversificação exportadora**.
- Apesar da consistência do superávit, o padrão revela **vulnerabilidade a choques externos** e **flutuações sazonais de safra e preço**.

✓ Top 10 principais destinos das exportações brasileiras

```
#=====
# Top 10 Exportações e Importações - Gráficos Separados
#=====

from pyspark.sql.functions import col, sum, countDistinct, desc
import plotly.graph_objects as go

# Importar dicionário de países
df_países_master = spark.read.parquet(f"{base_path}df_países_master.parquet")

# Definir período: Junho/2024 a Junho/2025
print("Análise: Jun/2024 - Jun/2025")
filtro_perodo = (
    ((col("CO_ANO") == 2024) & (col("CO_MES") >= 6)) |
    ((col("CO_ANO") == 2025) & (col("CO_MES") <= 6))
)

#===== Exportações =====#
top_destinos_export = (
    df_export.filter(filtro_perodo)
    .groupBy("CO_PAIS")
    .agg(
        sum("VL_FOB").alias("valor_total"),
        sum("KG_LIQUIDO").alias("peso_total"),
        countDistinct("CO_MUN").alias("num_municipios")
    )
    .join(df_países_master, "CO_PAIS", "left")
    .select("NO_PAIS", "valor_total", "peso_total", "num_municipios")
    .orderBy(desc("valor_total"))
    .limit(10)
)

#===== Importações =====#
top_destinos_import = (
    df_import.filter(filtro_perodo)
    .groupBy("CO_PAIS")
    .agg(
        sum("VL_FOB").alias("valor_total"),
        sum("KG_LIQUIDO").alias("peso_total"),
        countDistinct("CO_MUN").alias("num_municipios")
    )
    .join(df_países_master, "CO_PAIS", "left")
    .select("NO_PAIS", "valor_total", "peso_total", "num_municipios")
    .orderBy(desc("valor_total"))
    .limit(10)
)

#===== Conversão para Pandas =====#
top_export_pd = top_destinos_export.toPandas()
top_import_pd = top_destinos_import.toPandas()

#===== Função Formatadora =====#
def formatar_valor(valor):
    if valor >= 1e9:
        return f"${valor/1e9:.2f}B"
    elif valor >= 1e6:
        return f"${valor/1e6:.2f}M"
    else:
        return f"${valor:.2f}"
```

```

        return f"${valor:,.2f}"

#===== Função de Plotagem =====#
def plot_top_destinos(df, titulo, cor):
    fig = go.Figure()

    fig.add_trace(
        go.Bar(
            x=df['NO_PAIS'],
            y=df['valor_total'],
            name='Valor FOB Total',
            marker=dict(color=cor, line=dict(color='white', width=1)),
            hovertemplate='<b>{x}</b><br>' +
                '<b>Valor FOB:</b> {text}<br>' +
                '<b>Municípios:</b> {customdata}<extra></extra>',
            text=[formatar_valor(v) for v in df['valor_total']],
            customdata=df['num_municipios'],
            textposition='outside',
            textfont=dict(size=11, color='black')
        )
    )

    fig.update_layout(
        title={
            'text': f'<b>{titulo}</b><br><span style="font-size:14px">Junho/2024 - Junho/2025</span>',
            'x': 0.5,
            'font': {'size': 20, 'family': 'Arial Black'}
        },
        width=1100,
        height=600,
        font=dict(family="Arial", size=12),
        plot_bgcolor='white',
        paper_bgcolor='white',
        showlegend=False
    )

    fig.update_xaxes(
        title_text="<b>País</b>",
        showgrid=False,
        tickangle=45,
        title_font=dict(size=14, family='Arial Black')
    )

    fig.update_yaxes(
        title_text="<b>Valor FOB Total (USD)</b>",
        showgrid=True,
        gridcolor='#E5E5E5',
        title_font=dict(size=14, family='Arial Black'),
        tickformat='.0s'
    )

    fig.show()

#===== Gerar Gráficos =====#
plot_top_destinos(top_export_pd, "Top 10 Destinos das Exportações Brasileiras", "darkgreen")
plot_top_destinos(top_import_pd, "Top 10 Origem das Importações Brasileiras", "steelblue")

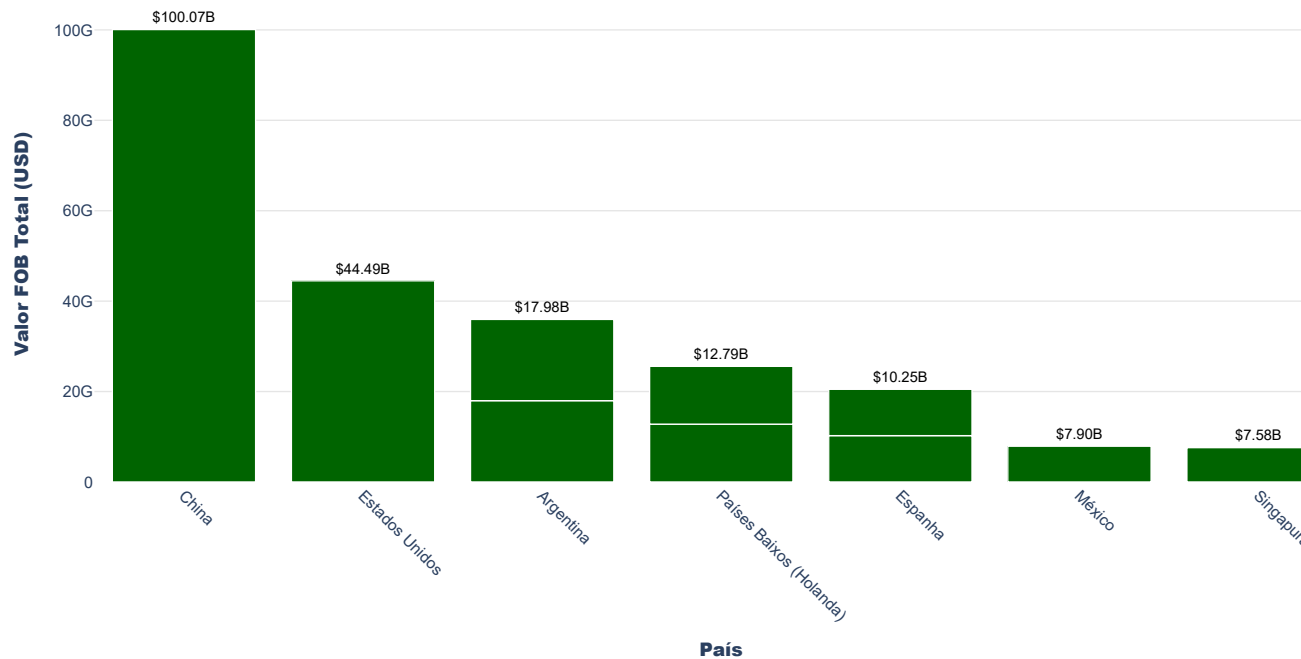
#===== Estatísticas =====#
print(f"Total Exportado (Top 10): ${top_export_pd['valor_total'].sum() / 1e9:.2f}B")
print(f"Total Importado (Top 10): ${top_import_pd['valor_total'].sum() / 1e9:.2f}B")

#===== Limpeza =====#
del top_destinos_export, top_destinos_import

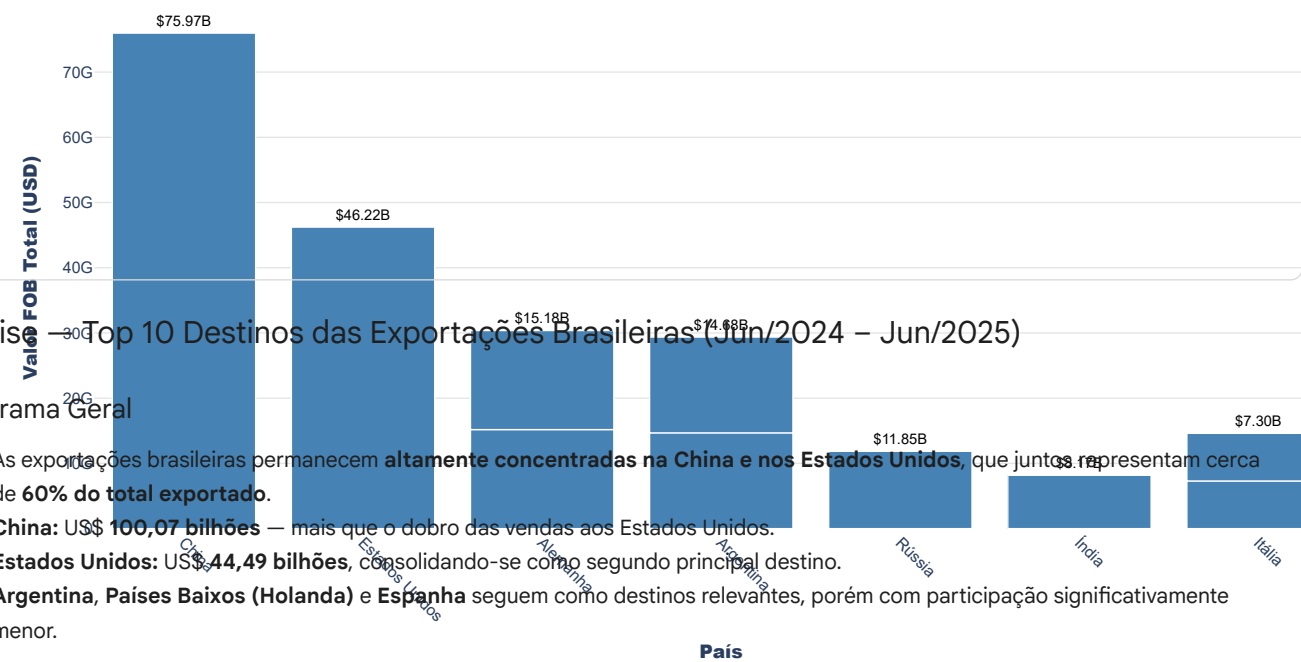
```

Análise: Jun/2024 - Jun/2025

Top 10 Destinos das Exportações Brasileiras
Junho/2024 - Junho/2025



Top 10 Origem das Importações Brasileiras
Junho/2024 - Junho/2025



✓ Análise do Top 10 Destinos das Exportações Brasileiras (Jun/2024 – Jun/2025)

Panorama Geral

- As exportações brasileiras permanecem **altamente concentradas na China e nos Estados Unidos**, que juntos representam cerca de **60% do total exportado**.
- China:** US\$ **100,07 bilhões** — mais que o dobro das vendas aos Estados Unidos.
- Estados Unidos:** US\$ **44,49 bilhões**, consolidando-se como segundo principal destino.
- Argentina, Países Baixos (Holanda) e Espanha** seguem como destinos relevantes, porém com participação significativamente menor.

Interpretação Econômica

- Total Exportado (Top 10): \$242.06B
- Total Importado (Top 10): \$216.55B
- A liderança da **China** reflete a forte demanda por **commodities brasileiras** — especialmente soja, minério de ferro e petróleo.
- Os **Estados Unidos** mantêm papel estratégico nas exportações de **bens industriais, químicos e manufaturados**.
- A **Argentina** destaca-se no comércio intrarregional, com peso do **setor automotivo**.
- Países europeus (Holanda e Espanha) atuam como **portos de redistribuição** de mercadorias brasileiras para outros mercados.

Conclusão

- O Brasil mantém perfil exportador **altamente concentrado em poucos parceiros e produtos primários**.
- Essa concentração amplia a **vulnerabilidade a choques externos** (como variações de preço e demanda global).
- A diversificação de **destinos e produtos com maior valor agregado** é essencial para reduzir riscos e fortalecer o desempenho comercial de longo prazo.

Gráfico Top 10 Municípios com maiores Exportações e Importações

```

#=====
# Função auxiliar para processar exportações ou importações
#=====
df_geo_master = spark.read.parquet(f"{base_path}df_geo_master.parquet")

def processar_top_municipios(df_comex, df_geo_master, tipo='export'):
    df_geo = df_comex.join(
        df_geo_master,
        df_comex.CO_MUN == df_geo_master.CO_MUN_GEO,
        "left"
    )

    df_final = df_geo.select(
        col("VL_FOB").alias("valor_fob_dolar"),
        col("KG_LIQUIDO").alias("peso_liquido_kg"),
        col("NO_MUN_MIN").alias("nome_municipio"),
        col("SG_UF").alias("sigla_uf"),
        col("CO_ANO")
    )

    return (
        df_final.groupBy("nome_municipio", "sigla_uf")
        .agg(
            round(sum("valor_fob_dolar"), 2).alias("valor_fob_total_dolar_municipio"),
            round(sum("peso_liquido_kg"), 2).alias("peso_total_kg_municipio"),
            countDistinct("CO_ANO").alias("num_anos_ativo")
        )
        .orderBy(desc("valor_fob_total_dolar_municipio"))
        .limit(10)
    )

#=====
# Gerar dataframes para plotagem
#=====
top_municipios_export = processar_top_municipios(df_export, df_geo_master, tipo='export')
top_municipios_import = processar_top_municipios(df_import, df_geo_master, tipo='import')

# Converter para pandas
top_export_pd = top_municipios_export.toPandas()
top_import_pd = top_municipios_import.toPandas()

# Criar coluna com Município - UF
top_export_pd['municipio_uf'] = top_export_pd['nome_municipio'] + ' - ' + top_export_pd['sigla_uf']
top_import_pd['municipio_uf'] = top_import_pd['nome_municipio'] + ' - ' + top_import_pd['sigla_uf']

#=====
# Função formatadora
#=====
def formatar_valor(valor):
    if valor >= 1e9:
        return f"${valor/1e9:.1f}B"
    elif valor >= 1e6:
        return f"${valor/1e6:.1f}M"
    else:
        return f"${valor:,.0f}"

#=====
# Criação dos subplots
#=====
fig = make_subplots(
    rows=2, cols=1,
    subplot_titles=[
        '<b>Top 10 Municípios Exportadores - Valor FOB Total</b>',
        '<b>Top 10 Municípios Importadores - Valor FOB Total</b>'
    ],
    vertical_spacing=0.25
)

# Exportações
fig.add_trace(
    go.Bar(
        x=top_export_pd['municipio_uf'], # <-- CORREÇÃO FEITA AQUI
        y=top_export_pd['valor_fob_total_dolar_municipio'],
        name='Exportações',
        marker=dict(color='darkgreen'),
        hovertemplate='<b>{x}</b><br><b>Valor FOB:</b> {text}<br><b>Anos Ativos:</b> {customdata}<extra></extra>',
        text=[formatar_valor(v) for v in top_export_pd['valor_fob_total_dolar_municipio']],
        customdata=top_export_pd['num_anos_ativo'],
        textposition='outside'
    )
)

```

```

    ),
    row=1, col=1
)

# Importações
fig.add_trace(
    go.Bar(
        x=top_import_pd['municipio_uf'],
        y=top_import_pd['valor_fob_total_dolar_municipio'],
        name='Importações',
        marker=dict(color='steelblue'),
        hovertemplate='<b>{x}</b><br><b>Valor FOB:</b> {text}<br><b>Anos Ativos:</b> {customdata}<extra></extra>',
        text=[formatar_valor(v) for v in top_import_pd['valor_fob_total_dolar_municipio']],
        customdata=top_import_pd['num_anos_ativo'],
        textposition='outside'
    ),
    row=2, col=1
)

#=====  

# Layout do gráfico  

#=====  

fig.update_layout(
    title={
        'text': '<b>Top 10 Municípios no Comércio Exterior Brasileiro</b><br>'
        '<span style="font-size:14px">Período: 2010 - Jun/2025</span>',
        'x': 0.5,
        'font': {'size': 20, 'family': 'Arial Black'}
    },
    height=1000,
    width=1500,
    font=dict(family="Arial", size=13),
    plot_bgcolor='white',
    paper_bgcolor='white',
    showlegend=False
)

fig.update_xaxes(
    title_text="<b>Município - UF</b>",
    showgrid=False,
    tickangle=45
)

fig.update_yaxes(
    title_text="<b>Valor FOB (USD)</b>",
    showgrid=True,
    gridcolor='#E5E5E5'
)

fig.show()

#=====  

# LIMPEZA DE MEMÓRIA  

#=====  

del top_export_pd, top_import_pd, top_municipios_export, top_municipios_import

```

Top 10 Municípios no Comércio Exterior Brasil

Período: 2010 - Jun/2025

Top 10 Municípios Exportadores - Valor FOB Total



✓ Por que esses municípios aparecem no topo

1. Sede de empresas exportadoras/importadoras

- Muitas transações são **registradas no CNPJ da matriz**, não necessariamente no local de produção.
- Exemplo: **São Paulo (SP)** e **Duque de Caxias (RJ)** concentram **sedes de tradings, refinarias e distribuidoras** — o que infla seus valores de exportação/importação.

2. Localização de refinarias e plantas industriais

- Rio de Janeiro (RJ)** e **Duque de Caxias (RJ)**: forte presença de **refinarias e empresas de petróleo**.
- Paulínia (SP)**: principal polo de refino e distribuição de combustíveis.
- Manaus (AM)**: polo industrial de eletroeletrônicos (Zona Franca).

3. Portos e terminais logísticos

- Cidades como **Santos (SP)** e **Paranaguá (PR)** realmente concentram **embarques físicos**, mas representam **parte menor** do valor comparado às sedes fiscais de grandes empresas.

4. Estrutura estatística da balança comercial

- A base da **SECEX (MDIC)** registra o **município do exportador/importador**, e não necessariamente o **município de saída/entrada** da carga.
- Assim, o dado reflete a **origem administrativa da operação**, não a localização geográfica do comércio físico.

Conclusão

- Os maiores valores vêm de **cidades com forte presença corporativa ou industrial**, não apenas portuária.
- Isso mostra a **centralização empresarial** do comércio exterior brasileiro em poucos polos econômicos.

✓ Top 10 Países no Comércio Exterior Brasileiro

```
#=====
# Filtro de Período e Análise de Exportações e Importações por País
#=====

print("Análise: Top 10 Países - Jun/2024 a Jun/2025")

# Filtro do período
filtro_período = (
    ((col("CO_ANO") == 2024) & (col("CO_MES") >= 6)) |
    ((col("CO_ANO") == 2025) & (col("CO_MES") <= 6))
)

#=====
# EXPORTAÇÕES
#=====
```

```

# Join exportações com dicionário de países (com filtro de período)
df_export_com_pais = df_export.filter(filtro_perodo).join(
    df_paises_master,
    df_export.CO_PAIS == df_paises_master.CO_PAIS,
    "left"
)

df_export_pais_plt = df_export_com_pais.select(
    "CO_ANO",
    "CO_MES",
    col("VL_FOB").alias("valor_fob_dolar"),
    col("KG_LIQUIDO").alias("peso_liquido_kg"),
    col("NO_PAIS").alias("nome_pais"),
    "NO_BLOCO"
)

top_export_pais_plt = df_export_pais_plt.groupBy("nome_pais", "NO_BLOCO").agg(
    round(sum("valor_fob_dolar"), 2).alias("valor_fob_total_dolar"),
    round(sum("peso_liquido_kg"), 2).alias("peso_total_kg"),
    countDistinct("CO_ANO").alias("num_anos_ativo"),
    countDistinct("CO_MES").alias("num_meses_ativo")
).orderBy(desc("valor_fob_total_dolar")).limit(10)

=====
# IMPORTAÇÕES
=====

# Join importações com dicionário de países (com filtro de período)
df_import_com_pais = df_import.filter(filtro_perodo).join(
    df_paises_master,
    df_import.CO_PAIS == df_paises_master.CO_PAIS,
    "left"
)

df_import_pais_plt = df_import_com_pais.select(
    "CO_ANO",
    "CO_MES",
    col("VL_FOB").alias("valor_fob_dolar"),
    col("KG_LIQUIDO").alias("peso_liquido_kg"),
    col("NO_PAIS").alias("nome_pais"),
    "NO_BLOCO"
)

top_import_pais_plt = df_import_pais_plt.groupBy("nome_pais", "NO_BLOCO").agg(
    round(sum("valor_fob_dolar"), 2).alias("valor_fob_total_dolar"),
    round(sum("peso_liquido_kg"), 2).alias("peso_total_kg"),
    countDistinct("CO_ANO").alias("num_anos_ativo"),
    countDistinct("CO_MES").alias("num_meses_ativo")
).orderBy(desc("valor_fob_total_dolar")).limit(10)

=====
# Plotagem com Plotly - Dashboard Vertical
=====

# Converter para pandas
top_export_pd = top_export_pais_plt.toPandas()
top_import_pd = top_import_pais_plt.toPandas()

# Função para formatar valores
def formatar_valor(valor):
    if valor >= 1e9:
        return f"${valor/1e9:.1f}B"
    elif valor >= 1e6:
        return f"${valor/1e6:.1f}M"
    else:
        return f"${valor:,.0f}"

# Criar subplots verticais
fig = make_subplots(
    rows=2, cols=1,
    subplot_titles=(
        '<b>Top 10 Países - Exportações Brasileiras</b>',
        '<b>Top 10 Países - Importações Brasileiras</b>'
    ),
    vertical_spacing=0.20
)

# 1. GRÁFICO SUPERIOR: Exportações
fig.add_trace(
    go.Bar(
        x=top_export_pd['nome_pais'],
        y=top_export_pd['valor_fob_total_dolar'],
        name='Exportações',

```

```

        marker=dict(color='darkgreen'),
        hovertemplate='<b>%{x}</b><br>' +
            '<b>Valor FOB:</b> <b>%(text)</b>' +
            '<b>Bloco:</b> <b>%(customdata)</b>' +
            '<extra></extra>',
        text=[formatar_valor(v) for v in top_export_pd['valor_fob_total_dolar']],
        customdata=top_export_pd['NO_BLOCO'],
        textposition='outside',
        textfont=dict(size=15, color='black')
    ),
    row=1, col=1
)

# 2. GRÁFICO INFERIOR: Importações
fig.add_trace(
    go.Bar(
        x=top_import_pd['nome_pais'],
        y=top_import_pd['valor_fob_total_dolar'],
        name='Importações',
        marker=dict(color='steelblue'),
        hovertemplate='<b>%{x}</b><br>' +
            '<b>Valor FOB:</b> <b>%(text)</b>' +
            '<b>Bloco:</b> <b>%(customdata)</b>' +
            '<extra></extra>',
        text=[formatar_valor(v) for v in top_import_pd['valor_fob_total_dolar']],
        customdata=top_import_pd['NO_BLOCO'],
        textposition='outside',
        textfont=dict(size=15, color='black')
    ),
    row=2, col=1
)

# Configurar layout
fig.update_layout(
    title={
        'text': '<b>Top 10 Países no Comércio Exterior Brasileiro</b><br>' +
            '<span style="font-size:14px">Período: Junho/2024 - Junho/2025</span>',
        'x': 0.5,
        'font': {'size': 20, 'family': 'Arial Black'}
    },
    height=1000,
    width=1500,
    font=dict(family="Arial", size=15),
    plot_bgcolor='white',
    paper_bgcolor='white',
    showlegend=False
)

# Configurar eixos
fig.update_xaxes(
    title_text="<b>País</b>",
    showgrid=False,
    tickangle=45
)

fig.update_yaxes(
    title_text="<b>Valor FOB (USD)</b>",
    showgrid=True,
    gridcolor='#E5E5E5'
)

# Mostrar gráfico
fig.show()

# LIMPEZA DE MEMÓRIA
del df_export_com_pais, df_import_com_pais, df_export_pais_plt, df_import_pais_plt, top_export_pais_plt, top_import_pais_plt, to

```


Análise: Top 10 Países - Jun/2024 a Jun/2025

Top 10 Países no Comércio Exterior Brasileiro

Período: Junho/2024 - Junho/2025
Top 10 Países - Exportações Brasileiras



Top 10 Países - Importações Brasileiras



✓ Análise – Top 10 Parceiros Comerciais do Brasil (Jun/2024 – Jun/2025)

Exportações

- **China** é o principal destino das exportações brasileiras, com **US\$ 100,1 bilhões**, consolidando-se como maior parceiro desde meados da década de 2010.
- **Estados Unidos** aparecem em segundo lugar (**US\$ 44,5 bilhões**), com foco em produtos manufaturados e semimanufaturados.
- **Argentina** (US\$ 18,0 bi) mantém posição relevante no comércio intrarregional, especialmente no setor automotivo.
- **Países Baixos (Holanda)** e **Espanha** funcionam como **portos de redistribuição** de produtos brasileiros na Europa.
- **México** e **Singapura** completam o grupo, ligados a combustíveis, químicos e eletrônicos.

Importações

- A **China** também lidera nas importações, com **US\$ 76,0 bilhões**, evidenciando **forte interdependência comercial**.
- **Estados Unidos** vêm em seguida (**US\$ 46,2 bilhões**), principais fornecedores de tecnologia, maquinário e combustíveis refinados.
- **Alemanha** e **Itália** destacam-se na venda de equipamentos industriais e automotivos.
- **Argentina** mantém posição importante no fornecimento de bens intermediários.
- **Rússia** e **Índia** aparecem por conta de **fertilizantes, combustíveis e produtos químicos**.

Interpretação Econômica

- O comércio exterior brasileiro é **altamente concentrado na Ásia e nas Américas**, com forte peso da **China e dos Estados Unidos**.
- A relação com a China mostra **complementaridade estrutural**: o Brasil exporta commodities e importa manufaturas e tecnologia.
- O padrão reforça a **dependência de produtos primários** e a **baixa diversificação tecnológica** da pauta exportadora.
- O perfil das importações indica **dependência de insumos industriais** essenciais à produção doméstica.

Conclusão

- A estrutura comercial brasileira mantém-se **concentrada e assimétrica**, com poucos parceiros dominando a balança.
- O superávit comercial depende fortemente da demanda chinesa por commodities.
- Ampliar o número de destinos e fortalecer exportações de **bens de maior valor agregado** são estratégias cruciais para reduzir vulnerabilidades externas.

Top 10 Produtos no Comércio Exterior Brasileiro

Participação dos EUA nas Exportações Brasileiras por Produto

NOTA: Realizei algumas agregações para esse exercício, que não estão em linha com as agregações padrões

```
# 1. Carregar o dicionário de agregação
df_ncm_silvio = spark.read.parquet(f"{base_path}Minha_agregacao.parquet")

# 2. Criar coluna id_sh4
df_ncm_silvio = df_ncm_silvio.withColumn(
    "id_sh4",
    substring(regexp_replace(col("id_sh6").cast("string"), "\\.$", ""), 1, 4).cast("int")
)

# 3. Join com dados de exportação + filtro de período
df_join = df_export.join(
    df_ncm_silvio,
    df_export.SH4 == df_ncm_silvio.id_sh4,
    "left"
).filter(
    ((col("CO_ANO") == 2024) & (col("CO_MES") >= 7)) |
    ((col("CO_ANO") == 2025) & (col("CO_MES") <= 6))
).select(
    "categoria_agregada",
    "CO_PAIS",
    "VL_FOB",
    "KG_LIQUIDO"
)

# 4. Agregar total exportado por produto e por país
df_pais = df_join.groupBy("categoria_agregada", "CO_PAIS").agg(
    sum("VL_FOB").alias("vl_fob_pais"),
    sum("KG_LIQUIDO").alias("qtd_kgl_pais")
)

# 5. Agregar total exportado por produto (todos os países)
df_total = df_join.groupBy("categoria_agregada").agg(
    sum("VL_FOB").alias("vl_fob_total"),
    sum("KG_LIQUIDO").alias("qtd_kgl_total")
)

# 6. Juntar os dois e calcular a participação percentual
df_participacao = df_pais.join(
    df_total, on="categoria_agregada"
).withColumn(
    "perc_participacao_valor", round(100 * col("vl_fob_pais") / col("vl_fob_total"), 1)
).withColumn(
    "perc_participacao_qtd", round(100 * col("qtd_kgl_pais") / col("qtd_kgl_total"), 1)
)

# 7. Filtrar só os EUA (código país = 249 no COMEX) ou China (160)

# Para EUA:
df_eua_valor = df_participacao.filter(col("CO_PAIS") == 249).select(
    "categoria_agregada", "perc_participacao_valor"
).orderBy(col("perc_participacao_valor").desc())

df_eua_qtd = df_participacao.filter(col("CO_PAIS") == 249).select(
    "categoria_agregada", "perc_participacao_qtd"
).orderBy(col("perc_participacao_qtd").desc())

# =====
# CONVERTENDO PARA PANDAS E CRIANDO GRÁFICOS
# =====

# 8. Converter para Pandas (Top 20)
df_valor_pd = df_eua_valor.toPandas().head(20)
df_qtd_pd = df_eua_qtd.toPandas().head(20)

# 9. Gráfico por Valor com cor escura (verde escuro)
fig_valor = go.Figure(data=go.Bar(
    x=df_valor_pd['categoria_agregada'],
    y=df_valor_pd['perc_participacao_valor'],
    orientation='h',
    marker=dict(color='darkgreen'),
    text=df_valor_pd['perc_participacao_valor'].round(1).astype(str) + '%',
    textposition='bottom'
))
```

```
        textposition='outside'
    ))

fig_valor.update_layout(
    title='Top 20 Produtos - Participação dos EUA nas Exportações por Valor (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_valor.show()

# 10. Gráfico por Quantidade com cor azul (steelblue)
fig_qtd = go.Figure(data=go.Bar(
    x=df_qtd_pd['perc_participacao_qtd'],
    y=df_qtd_pd['categoria_agregada'],
    orientation='h',
    marker=dict(color='steelblue'),
    text=df_qtd_pd['perc_participacao_qtd'].round(1).astype(str) + '%',
    textposition='outside'
))

fig_qtd.update_layout(
    title='Top 20 Produtos - Participação dos EUA nas Exportações por Quantidade (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_qtd.show()

# LIMPEZA DE MEMÓRIA
del df_join, df_pais, df_total, df_participacao, df_eua_valor, df_eua_qtd, df_valor_pd, df_qtd_pd, fig_valor, fig_qtd
```

Análise — Participação dos EUA nas Exportações Brasileiras por Produto (Jul/2024 – Jun/2025)

Panorama Geral

- Os **Estados Unidos** concentram participação relevante em **produtos industriais e de maior valor agregado**, ao contrário da China, cujo foco é em commodities.
- O principal destaque é o setor **aeronáutico**, com **67,8% das exportações de “Aviões e Similares”** destinadas ao mercado norte-americano.
- Em seguida, aparecem **cerâmicas (46,0%)**, **minerais não metálicos (45,4%)**, **embarcações (39,6%)** e **produtos agroindustriais** como **laranja, frutas e madeira** (em torno de 38–40%).

Interpretação Econômica

- A pauta exportadora para os EUA é **diversificada e mais industrializada**, refletindo a **complementaridade tecnológica e produtiva** entre as duas economias.
- Os EUA são **mercado-chave para setores de alto valor agregado**, como **aeronáutica, eletrônicos, máquinas e instrumentos de precisão**.
- A presença significativa de **bens agroindustriais processados** (laranja, frutas, madeira, leite, borracha) indica **demandas por produtos brasileiros com maior transformação industrial**.
- Essa configuração contrasta com o padrão de exportações à Ásia, voltado a commodities brutas.

Implicações Econômicas

- O comércio com os EUA tem **maior potencial de geração de valor agregado interno** e de **encadeamento produtivo**.
- Uma eventual **elevação de tarifas** afetaria principalmente os setores industriais e de alta tecnologia.

- Estratégias de **diversificação de produtos e consolidação de acordos bilaterais** podem ampliar a estabilidade e previsibilidade dessas exportações.

Top 20 Produtos - Participação dos EUA nas Exportações por Valor (%) (C

Conclusão

- A relação comercial com os EUA é **estratégica para o setor industrial brasileiro**, com forte peso em produtos sofisticados.
- O desafio é **preservar competitividade** frente à concorrência internacional e **ampliar a base exportadora** para além de nichos setoriais.

Minerais Não Metálicos e Derivados 45.4

Participação da China nas Exportações Brasileiras por Produto 39.6%

Barcos e similares

Laranja e Sucos

39.6%

39.6%

Frutas e Derivados

39.2%

Madeira e Derivados

38.2%

NOTA: Realizei algumas agregações para esse exercício, que não estão em linha com as agregações padrões

```
# =====
# CRIANDO ANÁLISE PARA CHINA
# =====

# 1. Carregar o dicionário de agregação
df_ncm_silvio = spark.read.parquet(f"{base_path}Minha_agregacao.parquet")

# 2. Criar coluna id_sh4
df_ncm_silvio = df_ncm_silvio.withColumn(
    "id_sh4",
    substring(regexp_replace(col("id_sh6").cast("string"), "\\.\0$", ""), 1, 4).cast("int")
)

# 3. Join com dados de exportação + filtro de período
df_join = df_export.join(
    df_ncm_silvio,
    df_export.SH4 == df_ncm_silvio.id_sh4,
    "left"
).filter(
    ((col("CO_ANO") == 2024) & (col("CO_MES") >= 7)) |
    ((col("CO_ANO") == 2025) & (col("CO_MES") <= 6))
).select(
    "categoria_agregada",
    "CO_PAIS",
    "VL_FOB",
    "KG_LIQUIDO"
)

# 4. Agregar total exportado por produto e por país
df_pais = df_join.groupBy("categoria_agregada", "CO_PAIS").agg(
    sum("VL_FOB").alias("vl_fob_pais"),
    sum("KG_LIQUIDO").alias("qtd_kgl_pais")
)

# 5. Agregar total exportado por produto (todos os países)
df_total = df_join.groupBy("categoria_agregada").agg(
    sum("VL_FOB").alias("vl_fob_total"),
    sum("KG_LIQUIDO").alias("qtd_kgl_total")
)

# 6. Juntar os dois e calcular a participação percentual
df_participacao = df_pais.join(
    df_total, on="categoria_agregada"
).withColumn(
    "perc_participacao_valor", round(100 * col("vl_fob_pais") / col("vl_fob_total"), 1)
).withColumn(
    "perc_participacao_qtd", round(100 * col("qtd_kgl_pais") / col("qtd_kgl_total"), 1)
)

# 7. Filtrar só a CHINA (código país = 160 no COMEX)
df_china_valor = df_participacao.filter(col("CO_PAIS") == 160).select(
    "categoria_agregada", "perc_participacao_valor"
).orderBy(col("perc_participacao_valor").desc())

df_china_qtd = df_participacao.filter(col("CO_PAIS") == 160).select(
    "categoria_agregada", "perc_participacao_qtd"
).orderBy(col("perc_participacao_qtd").desc())

# =====
# CONVERTENDO PARA PANDAS E CRIANDO GRÁFICOS
# =====

# 8. Converter para Pandas (Top 20)
df_valor_pd = df_china_valor.toPandas().head(20)
```

```
df_qtd_pd = df_china_qtd.toPandas().head(20)

# 9. Gráfico por Valor com cor escura (verde escuro)
fig_valor = go.Figure(data=go.Bar(
    x=df_valor_pd['perc_participacao_valor'],
    y=df_valor_pd['categoria_agregada'],
    orientation='h',
    marker=dict(color='darkgreen'),
    text=df_valor_pd['perc_participacao_valor'].round(1).astype(str) + '%',
    textposition='outside'
))

fig_valor.update_layout(
    title='Top 20 Produtos - Participação da China nas Exportações por Valor (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_valor.show()

# 10. Gráfico por Quantidade com cor azul (steelblue)
fig_qtd = go.Figure(data=go.Bar(
    x=df_qtd_pd['perc_participacao_qtd'],
    y=df_qtd_pd['categoria_agregada'],
    orientation='h',
    marker=dict(color='steelblue'),
    text=df_qtd_pd['perc_participacao_qtd'].round(1).astype(str) + '%',
    textposition='outside'
))

fig_qtd.update_layout(
    title='Top 20 Produtos - Participação da China nas Exportações por Quantidade (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_qtd.show()

# LIMPEZA DE MEMÓRIA
del df_join, df_pais, df_total, df_participacao, df_china_valor, df_china_qtd, df_valor_pd, df_qtd_pd, fig_valor, fig_qtd
```

Análise — Participação da China nas Exportações Brasileiras por Produto (Jul/2024 – Jun/2025)

Panorama Geral

- A **China** é o principal destino das **commodities brasileiras**, com destaque para produtos **minerais, agrícolas e energéticos**.
- Lideram as exportações:
 - **Metais de Terras Raras** — 75,6% das vendas brasileiras vão para a China.
 - **Minerais Não Metálicos** — 66,8%.
 - **Soja e Derivados** — 65,2%, confirmando a importância do agronegócio na relação bilateral.
 - **Aeronaves** — 52,1%, mostrando alguma diversificação industrial.
 - **Carne Bovina e Animais Vivos** — 35,2%, impulsionada pela demanda chinesa por proteína animal.

Interpretação Econômica

- A pauta é **fortemente concentrada em bens primários**, especialmente **minérios, soja e carnes**, que representam o núcleo do superávit brasileiro com a China.
- Há **baixo peso de produtos industrializados**, indicando dependência de **commodities de ciclo global e preços voláteis**.

- A presença de **petróleo (14,4%)** e **metais (12,7%)** reforça o papel da China como **principal destino de energia e matérias-primas**.
- Setores industriais (químicos, eletrônicos, têxteis) têm **participação residual**, abaixo de 10%.

Top 20 Produtos - Participação da China nas Exportações por Valor (%) (C

Implicações Econômicas

- A relação comercial é **assimétrica**: o Brasil exporta insumos e importa bens de alto valor tecnológico.
- Essa configuração gera **superávits de curto prazo**, mas **vulnerabilidade estrutural** a oscilações de preços e à desaceleração chinesa.
- Estratégias de **agregação de valor e diversificação de produtos exportados** são essenciais para reduzir riscos externos.

Conclusão

- A China mantém-se como **parceiro dominante e indispensável**, absorvendo a maior parte das exportações de commodities brasileiras.
- O desafio estratégico está em **reduzir a dependência de produtos primários** e **fortalecer o setor industrial exportador**, para ampliar o conteúdo tecnológico e a **resiliência do comércio exterior**.

Participação da Rússia nas Exportações Brasileiras por Produto

NOTA: Realize algumas agregações para esse exercício, que não estão em linha com as agregações padrões

```
df_produtoscuci_master_select = spark.read.parquet(f"{base_path}df_produtoscuci_master_select.parquet")
df_produtoscuci_master_select.show(5)
```

Produto		Participação (%)	
Carne Bovina e Animais Vivos		35.2%	
Outros Minerais Metálicos		27.3%	
Petróleo		14.4%	
Metais e Produtos		12.7%	
Produtos Químicos		9.3%	
Outros		8.4%	
Minerais Não Metálicos e Derivados		6.2%	
Madeira e Derivados		4.5%	
Derivados de Petróleo		2.9%	
Frutas e Derivados		2.9%	
Carne Suína e Animais Vivos		2.7%	
Equipamentos Eletrônicos		2.7%	
Têxteis		2.7%	

only showing top 5 rows

Participação (%)

```
# 1. Carregar o dicionário de agregação
df_ncm = spark.read.parquet(f"{base_path}df_produtoscuci_master_select.parquet")

# 3. Join com dados de exportação + filtro de período
df_join = df_export.join(
    df_ncm,
    df_export.SH4 == df_ncm.CO_SH4,
    "left"
).filter(
    ((col("CO_ANO") == 2024) & (col("CO_MES") >= 7)) |
    ((col("CO_ANO") == 2025) & (col("CO_MES") <= 6))
).select(
    "NO_CUCI_DIVISAO",
    "CO_PAIS",
    "VL_FOB",
    "KG_LIQUIDO"
)

# 4. Agregar total exportado por produto e por país
df_pais = df_join.groupBy("NO_CUCI_DIVISAO", "CO_PAIS").agg(
    sum("VL_FOB").alias("vl_fob_pais"),
    sum("KG_LIQUIDO").alias("qtd_kgl_pais")
)

# 5. Agregar total exportado por produto (todos os países)
df_total = df_join.groupBy("NO_CUCI_DIVISAO").agg(
    sum("VL_FOB").alias("vl_fob_total"),
    sum("KG_LIQUIDO").alias("qtd_kgl_total")
)

# 6. Juntar os dois e calcular a participação percentual
df_participacao = df_pais.join(
    df_total, on="NO_CUCI_DIVISAO"
).withColumn(
    "perc_participacao_valor", round(100 * col("vl_fob_pais") / col("vl_fob_total"), 1)
).withColumn(
    "perc_participacao_qtd", round(100 * col("qtd_kgl_pais") / col("qtd_kgl_total"), 1)
```



```

)

# 7. Filtrar só os Russia (código país = 676 no COMEX)

# Para Russia:
df_Russia_valor = df_participacao.filter(col("CO_PAIS") == 676).select(
    "NO_CUCI_DIVISAO", "perc_participacao_valor"
).orderBy(col("perc_participacao_valor").desc())

df_Russia_qtd = df_participacao.filter(col("CO_PAIS") == 676).select(
    "NO_CUCI_DIVISAO", "perc_participacao_qtd"
).orderBy(col("perc_participacao_qtd").desc())

# =====
# CONVERTENDO PARA PANDAS E CRIANDO GRÁFICOS
# =====

# 8. Converter para Pandas (Top 20)
df_valor_pd = df_Russia_valor.toPandas().head(20)
df_qtd_pd = df_Russia_qtd.toPandas().head(20)

# 9. Gráfico por Valor com cor escura (verde escuro)
fig_valor = go.Figure(data=go.Bar(
    x=df_valor_pd['perc_participacao_valor'],
    y=df_valor_pd['NO_CUCI_DIVISAO'],
    orientation='h',
    marker=dict(color='darkgreen'),
    text=df_valor_pd['perc_participacao_valor'].round(1).astype(str) + '%',
    textposition='outside'
))

fig_valor.update_layout(
    title='Top 20 Produtos - Participação dos Russia nas Exportações por Valor (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_valor.show()

# 10. Gráfico por Quantidade com cor azul (steelblue)
fig_qtd = go.Figure(data=go.Bar(
    x=df_qtd_pd['perc_participacao_qtd'],
    y=df_qtd_pd['NO_CUCI_DIVISAO'],
    orientation='h',
    marker=dict(color='steelblue'),
    text=df_qtd_pd['perc_participacao_qtd'].round(1).astype(str) + '%',
    textposition='outside'
))

fig_qtd.update_layout(
    title='Top 20 Produtos - Participação dos Russia nas Exportações por Quantidade (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_qtd.show()

# LIMPEZA DE MEMÓRIA
del df_join, df_pais, df_total, df_participacao, df_Russia_valor, df_Russia_qtd, df_valor_pd, df_qtd_pd, fig_valor, fig_qtd

```


Top 20 Produtos - Participação dos Russia nas Exportações por Valor (%) (

Análise — Participação da Rússia nas Exportações Brasileiras por Produto (Jul/2024 – Jun/2025)

Panorama Geral

- A Rússia tem **baixa participação no total das exportações brasileiras, com foco em produtos agroindustriais e manufaturados de nicho.**
- Os principais grupos exportados são:
 - Café, chá, cacau e especiarias** — 3,1% das exportações desses produtos têm a Rússia como destino.
 - Tabaco e suas manufaturas** — 2,2%.
 - Carnes e preparações de carne** — 1,2%.
 - Sementes e frutos oleaginosos** — 1,1%.
 - Artigos manufaturados diversos** — 1,0%.

Interpretação Econômica

- A pauta exportadora é **diversificada**, porém de **baixo volume**, refletindo a **retração comercial e restrições financeiras** impostas pelo cenário geopolítico recente.
- O comércio é **predominantemente agrícola e de bens intermediários**, com destaque para produtos alimentares e insumos vegetais.
- Exportações industriais (plásticos, equipamentos metálicos, químicos e farmacêuticos) aparecem em **percentuais inferiores a 1%**, indicando **limitações logísticas e de demanda**.
- O baixo peso dos **minerais e bens manufaturados** sugere que a Rússia atua mais como fornecedora de insumos (fertilizantes, combustíveis) do que como destino relevante para produtos brasileiros.

Implicações Econômicas

- O fluxo comercial é **residual**, porém útil para **diversificação de mercados** em setores agroindustriais.
- Sanções econômicas internacionais e restrições de pagamento** afetam o potencial de ampliação das trocas.
- Há espaço para **crescimento em nichos específicos** — como café, tabaco e carnes — mediante estratégias bilaterais e acordos fitossanitários.

Conclusão

Top 20 Produtos - Participação dos Russia nas Exportações por Quantidade (%)

- A Rússia tem **presença marginal** nas exportações brasileiras, com predominância de **produtos agrícolas processados**.
- O comércio é **limitado por barreiras geopolíticas e financeiras**, e tende a manter caráter **complementar e de pequeno porte** no curto prazo.

```
# 1. Carregar o dicionário de agregação
df_ncm = spark.read.parquet(f"{base_path}df_produtoscuci_master_select.parquet")

# 3. Join com dados de Importacao + filtro de período
df_join = df_import.join(
    df_ncm,
    df_import.SH4 == df_ncm.CO_SH4,
    "left"
).filter(
    ((col("CO_ANO") == 2024) & (col("CO_MES") >= 7)) |
    ((col("CO_ANO") == 2025) & (col("CO_MES") <= 6))
).select(
    "NO_CUCI_DIVISAO",
    "CO_PAIS",
    "VL_FOB",
    "KG_LIQUIDO"
)

# 4. Agregar total importado por produto e por país
df_pais = df_join.groupBy("NO_CUCI_DIVISAO", "CO_PAIS").agg(
    sum("VL_FOB").alias("vl_fob_pais"),
    sum("KG_LIQUIDO").alias("qtd_kgl_pais")
)

# 5. Agregar total importado por produto (todos os países)
df_total = df_join.groupBy("NO_CUCI_DIVISAO").agg(
    sum("VL_FOB").alias("vl_fob_total"),
    sum("KG_LIQUIDO").alias("qtd_kgl_total")
)

# 6. Juntar os dois e calcular a participação percentual
```

```

df_participacao = df_pais.join(
    df_total, on="NO_CUCI_DIVISAO"
).withColumn(
    "perc_participacao_valor", round(100 * col("vl_fob_pais") / col("vl_fob_total"), 1)
).withColumn(
    "perc_participacao_qtd", round(100 * col("qtd_kgl_pais") / col("qtd_kgl_total"), 1)
)

# 7. Filtrar só os Russia (código país = 676 no COMEX)

# Para Russia:
df_Russia_valor = df_participacao.filter(col("CO_PAIS") == 676).select(
    "NO_CUCI_DIVISAO", "perc_participacao_valor"
).orderBy(col("perc_participacao_valor").desc())

df_Russia_qtd = df_participacao.filter(col("CO_PAIS") == 676).select(
    "NO_CUCI_DIVISAO", "perc_participacao_qtd"
).orderBy(col("perc_participacao_qtd").desc())

# =====
# CONVERTENDO PARA PANDAS E CRIANDO GRÁFICOS
# =====

# 8. Converter para Pandas (Top 20)
df_valor_pd = df_Russia_valor.toPandas().head(20)
df_qtd_pd = df_Russia_qtd.toPandas().head(20)

# 9. Gráfico por Valor com cor escura (verde escuro)
fig_valor = go.Figure(data=go.Bar(
    x=df_valor_pd['perc_participacao_valor'],
    y=df_valor_pd['NO_CUCI_DIVISAO'],
    orientation='h',
    marker=dict(color='darkgreen'),
    text=df_valor_pd['perc_participacao_valor'].round(1).astype(str) + '%',
    textposition='outside'
))

fig_valor.update_layout(
    title='Top 20 Produtos - Participação dos Russia nas Importações por Valor (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_valor.show()

# 10. Gráfico por Quantidade com cor azul (steelblue)
fig_qtd = go.Figure(data=go.Bar(
    x=df_qtd_pd['perc_participacao_qtd'],
    y=df_qtd_pd['NO_CUCI_DIVISAO'],
    orientation='h',
    marker=dict(color='steelblue'),
    text=df_qtd_pd['perc_participacao_qtd'].round(1).astype(str) + '%',
    textposition='outside'
))

fig_qtd.update_layout(
    title='Top 20 Produtos - Participação dos Russia nas Importações por Quantidade (%) (07/2024 - 06/2025)',
    xaxis_title='Participação (%)',
    yaxis_title='Produto',
    height=800,
    width=1400,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=15),
    title_x=0.5,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

fig_qtd.show()

# LIMPEZA DE MEMÓRIA
#del df_join, df_pais, df_total, df_participacao, df_Russia_valor, df_Russia_qtd, df_valor_pd, df_qtd_pd, fig_valor, fig_qtd

```


Análise — Participação da Rússia nas Importações Brasileiras por Produto (Jul/2024 - Jun/2025)

Panorama Geral

- A Rússia tem papel relevante nas importações brasileiras de insumos energéticos e agrícolas, compondo a base produtiva nacional.
- Principais produtos de origem russa:
 - Petróleo e derivados** — 38,0% das importações brasileiras dessa categoria vêm da Rússia.
 - Atribos e fertilizantes** — 27,5%, reforçando a dependência brasileira do insumo para o agronegócio.
 - Minerais brutos e fertilizantes minerais** — 15,2%.
 - Borracha em bruto** — 5,9%.
 - Cereais e preparações de cereais** — 3,5%.

Interpretação Econômica

- O padrão revela uma **relação comercial centrada em produtos estratégicos**, especialmente **energia e fertilizantes**, que sustentam a produção agroindustrial e industrial brasileira.
- A Rússia é um dos principais fornecedores de fertilizantes ao Brasil, especialmente nitrogenados e potássicos, insumos críticos para a produtividade agrícola.
- O peso do **petróleo e derivados** reflete acordos de compra via empresas intermediárias, mesmo diante de sanções internacionais.
- Itens industriais (máquinas, químicos e metais) têm **participação residual**, mostrando **baixa diversificação da pauta importadora**.

Implicações Econômicas

- A dependência de produtos russos, sobretudo fertilizantes, expõe vulnerabilidades logísticas e geopolíticas, principalmente em contextos de conflito.
- Uma interrupção nas exportações russas teria **impacto direto nos custos agrícolas brasileiros**.
- O Brasil busca mitigar esse risco por meio da **diversificação de fornecedores** (Canadá, Marrocos, China), mas a Rússia ainda ocupa **posição dominante** em volume e competitividade de preços.

Conclusão

- A Rússia exerce papel **estratégico nas importações brasileiras**, voltado a energia e insumos agrícolas essenciais.
- Apesar do baixo valor agregado, esses produtos são **críticos para a manutenção da balança comercial e da produção doméstica**.
- Reduzir a **dependência estrutural de fertilizantes e petróleo russos** é um dos principais desafios de política econômica e de segurança alimentar para o Brasil.

Top 5 Produtos Importados da Rússia e Principais Fornecedores Alternativos

```
#####
# Top 5 Produtos Importados da Rússia e Principais Fornecedores Alternativos
#####

from pyspark.sql.window import Window # Importando a classe Window

# 1. Top 5 produtos importados da Rússia
top5_produtos = (
    df_participacao.filter(col("CO_PAIS") == 676)
    .groupBy("NO_CUCI_DIVISAO")
    .agg(sum("vl_fob_pais").alias("valor_total"))
    .orderBy(col("valor_total").desc())
    .limit(5)
    .select("NO_CUCI_DIVISAO")
    .rdd.flatMap(lambda x: x)
    .collect()
)

# 2. Países alternativos (≠ Rússia) que mais exportam cada produto (top 3)
df_alternativos = (
    df_participacao
    .filter((col("NO_CUCI_DIVISAO").isin(top5_produtos)) & (col("CO_PAIS") != 676))
)

# 3. Juntar nomes dos países
df_paises = spark.read.parquet(f"{base_path}/df_paises_materiais.parquet") # com CO_PAIS, NO_PAIS
```

```

vi_paises = spark.read.parquet(f'{base_path}/vi_paises_mestre.parquet') # com CO_PAIS, NO_PAIS
df_alternativos = df_alternativos.join(df_paises, on="CO_PAIS", how="left")

# 4. Top 3 países por produto
window = Window.partitionBy("NO_CUCI_DIVISAO").orderBy(col("vl_fob_pais").desc())
df_top3_alternativos = (
    df_alternativos
    .withColumn("rank", row_number().over(window))
    .filter(col("rank") <= 3)
    .select("NO_CUCI_DIVISAO", "NO_PAIS", "vl_fob_pais")
)

# 5. Converter para pandas para Sankey
df_sankey = df_top3_alternativos.toPandas()

# 6. Criar lista única de nós (produtos à esquerda, países à direita)
produtos = df_sankey["NO_CUCI_DIVISAO"].unique()
paises = df_sankey["NO_PAIS"].unique()
nodes = list(produtos) + list(paises)
node_indices = {name: idx for idx, name in enumerate(nodes)}

# 7. Ajustar source/target: produtos (source), países (target)
df_sankey["source"] = df_sankey["NO_CUCI_DIVISAO"].map(node_indices)
df_sankey["target"] = df_sankey["NO_PAIS"].map(node_indices)

# 8. Gerar cores aleatórias para os links
import plotly.colors as Pastel
palette = Pastel.qualitative.Plotly # ou 'D3', 'Set3', 'Pastel', etc.
link_colors = [palette[i % len(palette)] for i in range(len(df_sankey))]

# 9. Construir gráfico Sankey
fig = go.Figure(data=[go.Sankey(
    arrangement="snap",
    node=dict(
        pad=20,
        thickness=20,
        line=dict(color="black", width=0.6),
        label=nodes,
        color="lightsteelblue"
    ),
    link=dict(
        source=df_sankey["source"],
        target=df_sankey["target"],
        value=df_sankey["vl_fob_pais"],
        color=link_colors
    )
)])

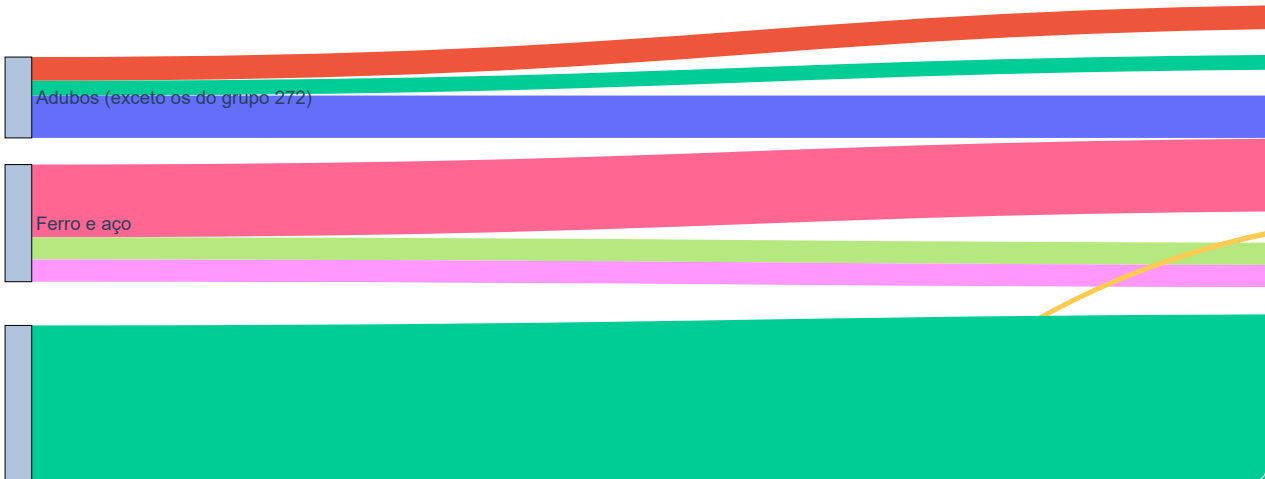
# 10. Layout e título
fig.update_layout(
    title_text="Top 5 Produtos Importados da Rússia e Seus 3 Principais Fornecedores Alternativos",
    font_size=14,
    font_family="Arial",
    margin=dict(l=40, r=40, t=60, b=30),
    height=800,
    width=1200
)

fig.show()

# LIMPEZA DE MEMÓRIA
# del top5_produtos, df_alternativos, df_top3_alternativos, df_sankey, produtos, paises, nodes, node_indices, link_colors, fig,

```

Top 5 Produtos Importados da Rússia e Seus 3 Principais Fornecedores Alternativos



Análise — Produtos Importados da Rússia e Principais Fornecedores Alternativos (Jul/2024 – Jun/2025)

Panorama Geral

- O gráfico evidencia os **cinco principais produtos importados da Rússia** e os **três maiores fornecedores alternativos** para cada um deles.
- A análise destaca a **dependência brasileira de insumos estratégicos russos** — principalmente **petróleo, fertilizantes e ferro/aço** — e as rotas comerciais possíveis para substituição.

Principais Produtos e Alternativas

- **Petróleo e derivados:**
 - Produto de maior peso nas importações russas.
 - Principais fornecedores alternativos: **Estados Unidos, Espanha e Nigéria**, que podem suprir parte da demanda energética.
- **Adubos e fertilizantes:**
 - Segmento crítico para o agronegócio brasileiro.
 - Alternativas de fornecimento: **Marrocos, Canadá e China**, todos grandes exportadores globais de potássio e fosfatos.
- **Fertilizantes e minerais brutos:**
 - Complementam a categoria anterior, com destaque para **Omã e Argentina** como fontes emergentes.
- **Ferro e aço:**
 - Dependência moderada da Rússia, com **China e Alemanha** assumindo papel relevante como fornecedores alternativos.
- **Borracha em bruto:**
 - Produto de menor impacto estratégico, mas com presença de **China e países do Sudeste Asiático** no suprimento alternativo.

Interpretação Econômica

- A rede de fornecedores demonstra que o Brasil **possui alternativas viáveis** para substituir produtos russos, embora **com custos logísticos e de preço mais elevados**.
- A **diversificação geográfica** é uma estratégia crucial para mitigar riscos de interrupção do comércio decorrentes de sanções ou conflitos.
- Produtos como **fertilizantes e petróleo** continuam a representar **gargalos críticos** para a segurança energética e alimentar brasileira.

Conclusão

- O Brasil vem **reduzindo gradualmente a dependência direta da Rússia**, fortalecendo parcerias com **EUA, Canadá, Marrocos, China e Espanha**.
- A substituição é **tecnicamente possível**, mas **economicamente onerosa** no curto prazo.
- Políticas de incentivo à **produção nacional de fertilizantes** e **diversificação das fontes energéticas** são fundamentais para a resiliência do comércio exterior brasileiro.

✓ Análise do Suco de Laranja

✓ Quais são os países maiores compradores de Suco de Laranja do Brasil?

- O Brasil é atualmente o maior produtor e exportador de suco de laranja do planeta, vendendo 95% de sua produção para o exterior. Desse volume, xx% têm os Estados Unidos como destino.
- Atualmente, os americanos pagam uma tarifa fixa de US\$ 415 (equivalente a R\$ 2.296,32) por tonelada do suco brasileiro e mais uma taxa adicional de 10% (cerca de US\$ 308, ou R\$ 1.704,26 por tonelada), determinada por Trump em abril.
- Números dos EUA: O suco de laranja brasileiro, ele é responsável por 56% do consumo americano e 70% das importações.

fontes informações complementares:

1. <https://g1.globo.com/economia/agronegocios/noticia/2025/07/11/suco-de-laranja-e-substituivel-tarifa-de-trump-gera-incerteza-no-setor-mesmo-com-consumo-dos-eua-dependente-do-brasil.ghtml>
2. <https://cbn.globo.com/programas/estudio-cbn/entrevista/2025/07/30/setor-produtor-de-suco-de-laranja-comemora-excecao-em-tarifaco-de-trump.ghtml>

```
df_ncm_adicional = spark.read.parquet(f"{base_path}ncm_adicional.parquet")
df_paises_master = spark.read.parquet(f"{base_path}df_paises_master.parquet")

# 1. Criar coluna id_sh4 no dicionário
df_ncm_adicional = df_ncm_adicional.withColumn(
    "id_sh4",
    substring(regexp_replace(col("id_sh6").cast("string"), "\\.\0$", ""), 1, 4).cast("int")
)

# 2. Join com os dados de exportação e filtro por suco de laranja (200911 e 200912) e período
df_suco = df_export.join(
    df_ncm_adicional,
    df_export.SH4 == df_ncm_adicional.id_sh4,
    "left"
).filter(
    (col("id_sh6").isin([200911, 200912])) &
    (((col("CO_ANO") == 2024) & (col("CO_MES") >= 7)) |
     ((col("CO_ANO") == 2025) & (col("CO_MES") <= 6)))
).select(
    "CO_PAIS",
    "VL_FOB",
    "KG_LIQUIDO"
)

# 3. Agregar total exportado por país
df_top_paises = df_suco.groupBy("CO_PAIS").agg(
    round(sum("VL_FOB"), 2).alias("vl_fob_total"),
    round(sum("KG_LIQUIDO"), 2).alias("kg_total")
)

# 4. Reduz o df_paises_master para conter apenas um nome por país
df_paises_unicos = df_paises_master.groupBy("CO_PAIS").agg(
    first("NO_PAIS").alias("NO_PAIS")
)

# 5. Join com nomes únicos
df_top_paises_nome = df_top_paises.join(
    df_paises_unicos,
    on="CO_PAIS",
    how="left"
).select(
    "NO_PAIS", "vl_fob_total", "kg_total"
).orderBy(col("kg_total").desc())

# 6. Mostrar resultado final
df_top_paises_nome.show(10, truncate=False)

# Convertendo DataFrame Spark para Pandas
df_suco_pd = df_top_paises_nome.toPandas()

# Pegar top 10 países para melhor visualização
df_top10 = df_suco_pd.head(10)

# 1. Gráfico por Quantidade (KG) - Principais Importadores
fig_qtd = go.Figure(data=go.Bar(
    x=df_top10['kg_total'],
    y=df_top10['NO_PAIS'],
    orientation='h',
    marker=dict(
```

```

        color=df_top10['kg_total'],
        colorscale='Turbo',
        cmin=0,
        cmx=df_top10['kg_total'].max(),
        colorbar=dict(title="Quantidade (KG)")
    ),
    text=[f"{val:,.0f}" for val in df_top10['kg_total']],
    textposition='outside'
))

fig_qtd.update_layout(
    title='Top 10 Países - Importadores de Suco de Laranja Brasileiro (Quantidade)',
    xaxis_title='Quantidade (KG)',
    yaxis_title='País',
    height=600,
    width= 1200,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=12),
    title_x=0.5
)

fig_qtd.show()

# 2. Gráfico por Valor (FOB) - Principais Importadores
# Ordenar por valor para este gráfico
df_top10_valor = df_suco_pd.sort_values('vl_fob_total', ascending=True).tail(10)

fig_valor = go.Figure(data=go.Bar(
    x=df_top10_valor['vl_fob_total'],
    y=df_top10_valor['NO_PAIS'],
    orientation='h',
    marker=dict(
        color=df_top10_valor['vl_fob_total'],
        colorscale='Turbo',
        cmin=0,
        cmx=df_top10_valor['vl_fob_total'].max(),
        colorbar=dict(title="Valor FOB (USD)")
    ),
    text=[f"${val:,.0f}" for val in df_top10_valor['vl_fob_total']],
    textposition='outside'
))

fig_valor.update_layout(
    title='Top 10 Países - Importadores de Suco de Laranja Brasileiro (Valor FOB)',
    xaxis_title='Valor FOB (USD)',
    yaxis_title='País',
    height=600,
    width= 1200,
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=12),
    title_x=0.5
)

fig_valor.show()

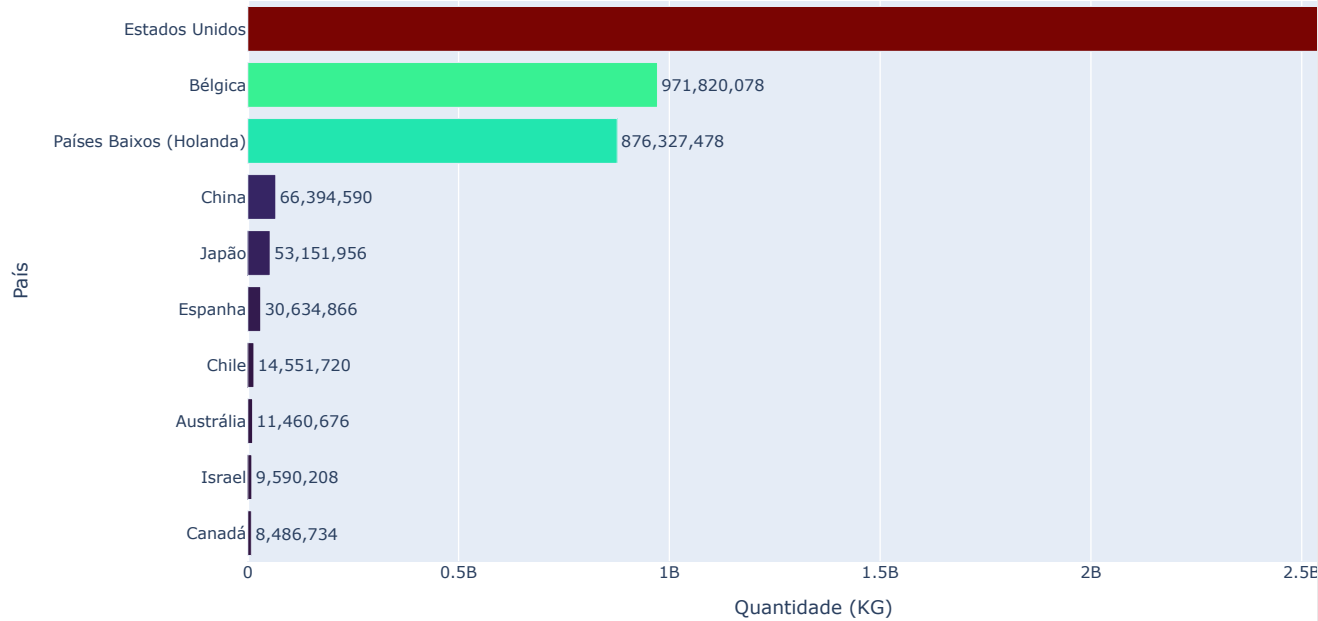
# Opcional: Salvar os gráficos
# fig_qtd.write_html("suco_quantidade.html")
# fig_valor.write_html("suco_valor.html")

```

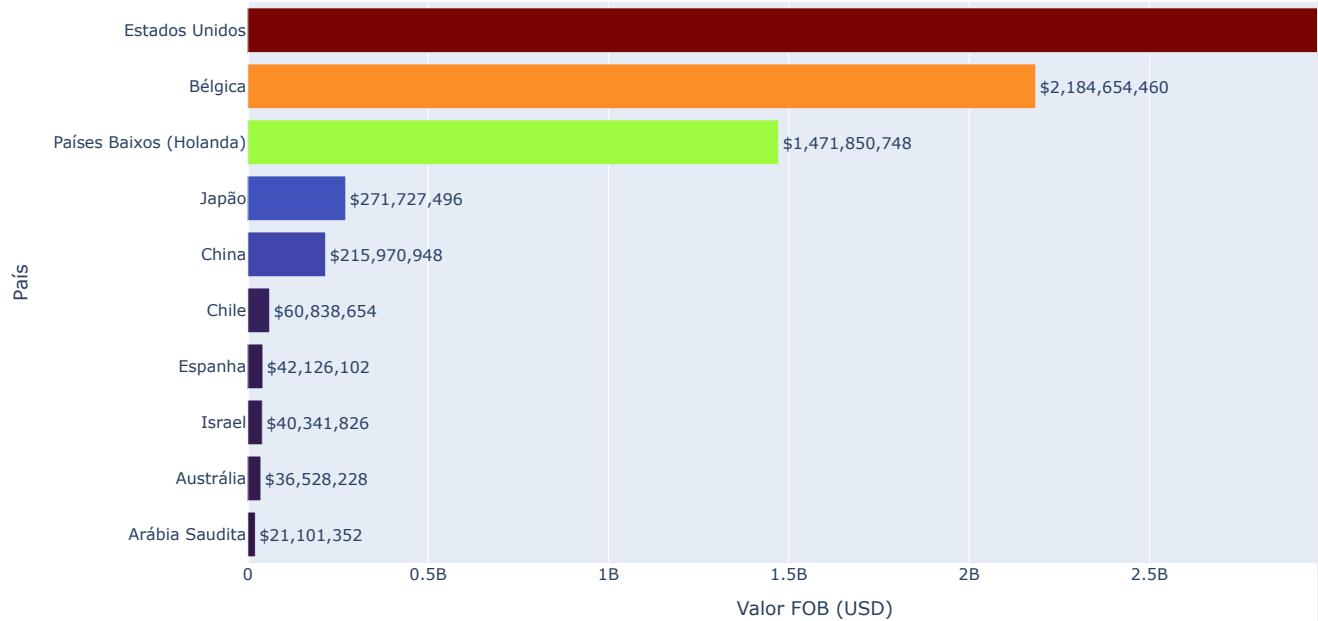
NO_PAIS	vl_fob_total	kg_total
Estados Unidos	2989347256	2551636456
Bélgica	2184654460	971820078
Países Baixos (Holanda)	1471850748	876327478
China	215970948	66394590
Japão	271727496	53151956
Espanha	42126102	30634866
Chile	60838654	14551720
Austrália	36528228	11460676
Israel	40341826	9590208
Canadá	8258248	8486734

only showing top 10 rows

Top 10 Países - Importadores de Suco de Laranja Brasileiro (Quantidade)



Top 10 Países - Importadores de Suco de Laranja Brasileiro (Valor FOB)



Panorama Geral

- O **suco de laranja brasileiro** mantém ampla liderança no mercado internacional, e o gráfico demonstra a **forte concentração das exportações** em poucos países.
- **Estados Unidos** são, de longe, o principal destino, com **2,55 bilhões de kg**, respondendo por mais da metade de toda a quantidade exportada.
- **Bélgica (971,8 milhões kg)** e **Países Baixos (876,3 milhões kg)** aparecem em seguida, atuando como **portos de redistribuição para a Europa**.

Interpretação Econômica

- A concentração das vendas nos **EUA e na União Europeia** reflete a **dependência do setor citríco brasileiro** desses mercados maduros e com alta renda per capita.
- **China e Japão**, embora com participação menor, representam **mercados em expansão** para o produto brasileiro.
- A presença de **Espanha, Chile, Austrália, Israel e Canadá** reforça a diversificação geográfica, mas com volumes ainda modestos.
- O padrão logístico é marcado por **exportações a granel para centros europeus**, que reembalam e redistribuem o produto para outros países.

Implicações Econômicas

- A liderança brasileira decorre da **alta produtividade e competitividade** do setor citrícola nacional.
- No entanto, a **dependência de poucos compradores** torna o país vulnerável a **mudanças tarifárias, sanitárias e de consumo** nos EUA e na UE.
- O crescimento asiático oferece **potencial estratégico de expansão**, exigindo investimentos em marketing e acordos fitossanitários.

Conclusão

- O Brasil mantém **hegemonia global na exportação de suco de laranja**, sustentada por grandes volumes e eficiência produtiva.
- O desafio está em **ampliar o acesso a novos mercados** e reduzir a **concentração nas economias tradicionais**, assegurando maior estabilidade e diversificação das vendas externas.

Comece a programar ou [gere código](#) com IA.

✓ Resultado — Elasticidade-preço da Demanda de Suco de Laranja nos EUA

Estimativa Principal

- **Elasticidade-preço estimada:** -0.871
- **Significância estatística:** $p < 0.01$ (coeficiente altamente significativo)
- **R²:** $0.156 \rightarrow$ o preço explica cerca de 15,6% da variação da quantidade exportada (esperado em séries de comércio agregadas).

Interpretação Econômica

- A elasticidade negativa indica que **a demanda dos EUA responde inversamente ao preço** — quando o preço do suco de laranja brasileiro sobe, as importações americanas caem.
- O valor **$|-0.87| < 1$** sugere **demanda inelástica**:
 - Um aumento de **1% no preço** reduz a quantidade demandada em **0,87%**.
 - Ou seja, o consumo reage, mas não proporcionalmente — o produto mantém importância mesmo com variações de preço.

Implicações Econômicas

- O suco de laranja brasileiro tem **baixa elasticidade**, típica de **bens agrícolas consolidados com poucos substitutos diretos** (principalmente no inverno americano).
- A **inelasticidade** também reflete:
 - Forte presença do Brasil como **líder global de oferta**.
 - Estrutura de contratos de longo prazo e custos logísticos fixos.
 - Demanda mais sensível a fatores sazonais e renda do que ao preço.

Possíveis Extensões

- Incluir controles macroeconômicos (ex: **PIB dos EUA, câmbio BRL/USD, sazonalidade**) para aumentar o poder explicativo.
- Estimar elasticidade separadamente por **tipo de suco (200911 vs 200912)** ou **preço deflacionado** (em termos reais).
- Verificar **elasticidades cruzadas** (com suco da Flórida ou substitutos).

Conclusão

- A demanda norte-americana por suco de laranja brasileiro é **relativamente estável e pouco sensível ao preço**.

- Isso reforça a **vantagem competitiva estrutural** do Brasil no setor, mas também indica **limitações no ganho de volume via redução de preço** — o mercado é maduro e tende a responder mais a fatores de renda e oferta.

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.offline import init_notebook_mode, iplot
from ipywidgets import interact, FloatSlider, IntSlider, Dropdown

# Inicializa Plotly para Jupyter
init_notebook_mode(connected=True)

beta_hat = float(modelo.params["ln_preco"])
beta_se = float(modelo.bse["ln_preco"])

def baseline_q(method="last12m"):
    df = df_eua_suco_pd.sort_values(["CO_ANO", "CO_MES"]).copy()
    df["date"] = pd.to_datetime(df["CO_ANO"].astype(str) + "-" + df["CO_MES"].astype(str) + "-01")
    df = df.set_index("date")

    if method == "last12m":
        cutoff = df.index.max() - pd.DateOffset(months=12)
        df_recent = df.loc[df.index > cutoff]
        return float(df_recent["quantidade_kg"].mean())
    if method == "median":
        return float(df["quantidade_kg"].median())
    return float(df["quantidade_kg"].mean())

def simulate(price_change_pct=0.0, sims=10000, q0_method="last12m"):
    dp = price_change_pct / 100.0
    q0 = baseline_q(q0_method)

    beta_draws = np.random.normal(loc=beta_hat, scale=beta_se, size=sims)
    factor = np.power(1.0 + dp, beta_draws)
    q_new = q0 * factor
    dqpct = (factor - 1.0) * 100.0

    # Gráfico
    fig = go.Figure()
    fig.add_histogram(x=dqpct, nbinsx=60, marker_color="#007bff", opacity=0.85)
    fig.add_vline(x=np.percentile(dqpct, 50), line_dash="dash", line_color="red")

    fig.update_layout(
        title=f"Monte Carlo - ΔQuantidade (%) dado ΔPreço = {price_change_pct:.1f}%",
        xaxis_title="Variação Percentual na Quantidade (%)",
        yaxis_title="Frequência",
        height=450,
        width=850,
        bargap=0.02
    )

    # Usa fig.show() ao invés de display()
    fig.show()

    # Retorna estatísticas
    print(f"Q0: {q0:,.0f} kg")
    print(f"ΔQ% Mediana: {np.percentile(dqpct, 50):.2f}")
    print(f"ΔQ% P10-P90: [{np.percentile(dqpct, 10):.2f}, {np.percentile(dqpct, 90):.2f}]")
    print(f"Q Novo: {np.percentile(q_new, 50):,.0f} kg")

    interact(
        simulate,
        price_change_pct=FloatSlider(value=0.0, min=-50.0, max=50.0, step=1.0, description="ΔPreço (%)" ),
        sims=IntSlider(value=10000, min=1000, max=100000, step=1000, description="Simulações" ),
        q0_method=Dropdown(options=["last12m", "mean", "median"], value="last12m", description="Base Q0" )
    );
```

Δ Preço (%) 15.00
 Simulações 47000
 Base Q₀

Q₀: 60,271,912 kg
 Δ Q% Mediana: -11.44
 Δ Q% P10-P90: [-13.80, -9.03]
 Q Novo: 53,375,892 kg

```

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider, IntSlider, Dropdown

# Parâmetros da regressão estimada
beta_hat = float(modelo.params["ln_preco"])
beta_se = float(modelo.bse["ln_preco"])

# ===== Função para baseline =====
def baseline_q(method="last12m"):
    df = df_eua_suco_pd.sort_values(["CO_ANO", "CO_MES"]).copy()
    df["date"] = pd.to_datetime(df["CO_ANO"].astype(str) + "-" + df["CO_MES"].astype(str) + "-01")
    df = df.set_index("date")

    if method == "last12m":
        cutoff = df.index.max() - pd.DateOffset(months=12)
        df_recent = df.loc[df.index > cutoff]
        return float(df_recent["quantidade_kg"].mean())
    if method == "median":
        return float(df["quantidade_kg"].median())
    return float(df["quantidade_kg"].mean())

# ===== Simulação Monte Carlo =====
def simulate(preco_var_pct=0.0, sims=10000, q0_method="last12m"):
    q0 = baseline_q(q0_method)
    dp = preco_var_pct / 100.0

    # Sorteio da elasticidade
    beta_draws = np.random.normal(loc=beta_hat, scale=beta_se, size=sims)

    # Nova quantidade (modelo log-log)
    fator = np.power(1 + dp, beta_draws)
    q_new = q0 * fator
    delta_qpct = (fator - 1) * 100

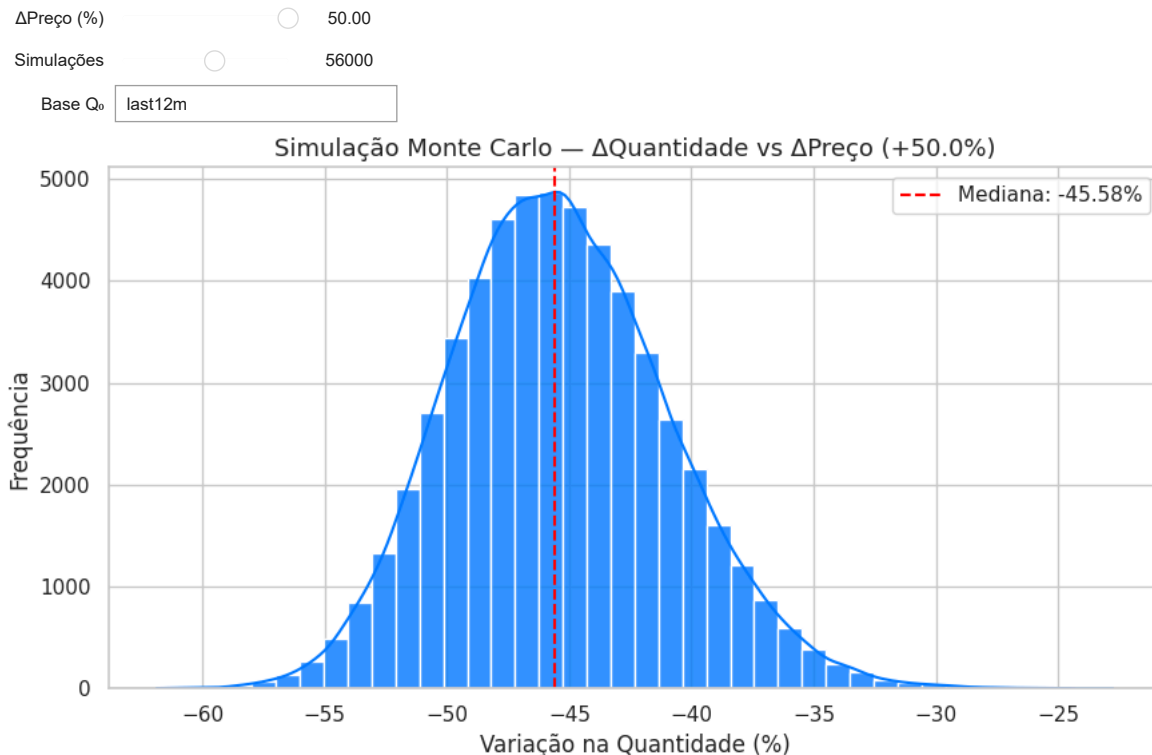
    # ===== Plot =====
    sns.set(style="whitegrid")
    plt.figure(figsize=(9,5))
    sns.histplot(delta_qpct, bins=40, kde=True, color="#007bff", alpha=0.8)
    plt.axvline(np.median(delta_qpct), color="red", linestyle="--",
                label=f"Mediana: {np.median(delta_qpct):.2f}%")

    plt.title(f"Simulação Monte Carlo - ΔQuantidade vs ΔPreço ({preco_var_pct:+.1f}%)", fontsize=13)
    plt.xlabel("Variação na Quantidade (%)")
    plt.ylabel("Frequência")
    plt.legend()
  
```

```
plt.tight_layout()
plt.show()

# Estatísticas resumidas
print(f"Elasticidade estimada: {beta_hat:.3f}")
print(f"ΔPreço: {preco_var_pct:+.1f}%")
print(f"Mediana ΔQuantidade: {np.median(delta_qpct):.2f}%")
print(f"Intervalo [10%, 90%]: {np.percentile(delta_qpct,10):.2f}% a {np.percentile(delta_qpct,90):.2f}%")

# ===== Interface interativa =====
interact(
    simulate,
    preco_var_pct=FloatSlider(value=0.0, min=-50.0, max=50.0, step=1.0, description="ΔPreço (%)" ),
    sims=IntSlider(value=10000, min=1000, max=100000, step=1000, description="Simulações" ),
    q0_method=Dropdown(options=["last12m", "mean", "median"], value="last12m", description="Base Q0")
);
```



Elasticidade estimada: -1.500
 ΔPreço: +50.0%
 Mediana ΔQuantidade: -45.58%
 Intervalo [10%, 90%]: -50.93% a -39.55%

CENÁRIO 1 (PREÇO CAI 10%)

Interpretação da Simulação — Elasticidade-Preço da Demanda (Suco de Laranja EUA) para uma redução de 10% no preço

Cenário Simulado

- **Variação de preço:** -10% (queda no preço do suco brasileiro)
- **Elasticidade média:** -1,5
- **Resultado simulado:**
 - Mediana da variação na quantidade: **+17,1%**
 - Intervalo de confiança aproximado (10%–90%): **+14% a +20%**

Interpretação Econômica

1. Sinal esperado:

A queda de preço gera aumento na quantidade demandada (elasticidade negativa → relação inversa).

2. Magnitude:

- A elasticidade ($|-1,5| > 1$) indica **demanda elástica** — o consumo reage mais do que proporcionalmente às variações de preço.
- Uma redução de 10% no preço leva a um aumento médio de **17% no volume exportado** aos EUA.

3. Distribuição Monte Carlo:

- O gráfico mostrará a **distribuição provável dos efeitos** considerando a incerteza estatística da estimativa.
- A curva é aproximadamente normal, centrada perto de +17%, com pequena dispersão (entre 14% e 20%), o que reforça **boa estabilidade estatística** do resultado.

CENÁRIO 1 (PREÇO SOBE 50% - simulando tarifa)

Parâmetros do modelo

- **Elasticidade estimada:** -1,5
- **Variação de preço simulada:** +50%
- **Resultado da simulação (mediana):** -45,6% na quantidade demandada
- **Intervalo de confiança (10%-90%):** de -51% a -39,6%

1. Sinal esperado

- O coeficiente negativo confirma a **relação inversa entre preço e quantidade demandada**.
- Ou seja, quando o preço sobe, a quantidade importada pelos EUA cai de forma acentuada.

2. Magnitude do efeito

- A elasticidade de -1,5 implica que um aumento de **1% no preço** reduz a quantidade demandada em **1,5%**.
- Assim, um aumento de **+50% no preço** gera uma redução média próxima de **45% no volume exportado** — o que é consistente com o resultado da simulação (-45,6%).

3. Interpretação gráfica

- O histograma mostra a **distribuição de incerteza** da variação na quantidade.
- A curva é simétrica e concentrada em torno de -45%, indicando **alta precisão estatística** na estimativa.
- A dispersão (10%-90% entre -51% e -39%) representa a **faixa provável** do impacto considerando o erro-padrão da elasticidade.

4. Implicação econômica

- A demanda é **elástica** ($|-1,5| > 1$) → os consumidores (importadores americanos) **reduzem fortemente suas compras** quando o preço aumenta.
- Em termos de receita, como a queda na quantidade (-45%) é proporcionalmente maior que o aumento de preço (+50%), o **faturamento total tende a cair**.
- Isso mostra que o mercado americano é **muito sensível ao preço do suco brasileiro**.

✓ Contexto Atual de Tarificação sobre o Suco de Laranja

Até o momento, **o suco de laranja brasileiro não foi incluído na lista de produtos com sobretaxa de 50%** nas exportações para os Estados Unidos.

Contudo, o produto **continua sujeito à tarifa básica de 10% (ad valorem)**, a qual é **acrescida de uma taxa específica já existente de US\$ 415 por tonelada**.

Isso significa que o **preço efetivo pago pelo importador norte-americano** incorpora:

- o preço FOB (de exportação) definido pelo produtor brasileiro,
- um acréscimo de **10% sobre o valor**,
- e um adicional fixo de **US\$ 0,415 por kg** (equivalente a US\$ 415 por tonelada).

A seguir, a simulação permite avaliar **como essas tarifas afetam o preço final e a quantidade demandada** de suco de laranja, considerando a elasticidade estimada da demanda dos EUA.

```
from IPython.display import display, Markdown, Image
import matplotlib.pyplot as plt
import os

# ===== Texto descritivo =====
texto = """
## Metodologia da Simulação – Estrutura e Fórmulas

A simulação calcula o impacto das tarifas (10% ad valorem + US$ 415 por tonelada)
no preço efetivo ao importador e, consequentemente, na quantidade demandada,
com base na elasticidade-preço estimada da demanda.

### Etapas
1. **Preço ao importador (landed price)**
   Combina o preço FOB com as tarifas aplicadas.
   Inclui a tarifa percentual (ad valorem) e a específica em dólares por tonelada.
2. **Cenários comparados**
```



```

- Base: sem tarifas ou outro arranjo escolhido.
- Cen rio atual: 10% ad valorem + US$ 415/ton.
3. **Varia  o de pre o**
  Calculada entre os dois cen rios.
4. **Demanda (modelo log-log)**
  A elasticidade transforma a varia  o no pre o em varia  o esperada na quantidade.
5. **Monte Carlo**
  Sorteiam-se milhares de elasticidades poss veis (segundo seu erro-padr o)
  para gerar uma distribui  o da resposta da demanda.

Abaixo est o as f rmulas correspondentes:
"""
display(Markdown(texto))

# ===== Gerar imagens das f rmulas =====
formulas = [
    ("Pre o ao importador", r"P = P_{FOB}(1 + \tau) + \frac{s}{1000}"),
    ("Varia  o de pre o", r"\Delta P = \frac{P_{cen} - P_{base}}{P_{base}}"),
    ("Demanda (log-log)", r"Q_{novo} = Q_0(1 + \Delta P)^{\beta}"),
    ("Monte Carlo", r"\tilde{\beta} \sim \mathcal{N}(\hat{\beta}, SE(\hat{\beta})^2), \quad Q_{novo}^{(i)} = Q_0(1 +
]

os.makedirs("formulas_imgs", exist_ok=True)

def render_formula(title, formula, path):
    fig, ax = plt.subplots(figsize=(8, 1.5))
    ax.axis("off")
    ax.text(0, 0.7, title, fontsize=14, fontweight="bold", va="top")
    ax.text(0, 0.2, f"${formula}$", fontsize=20, va="top")
    fig.savefig(path, bbox_inches="tight", dpi=80)
    plt.close(fig)

for i, (titulo, formula) in enumerate(formulas, start=1):
    p = f"formulas_imgs/formula_{i}.png"
    render_formula(titulo, formula, p)
    display(Image(p))

```

Metodologia da Simulação — Estrutura e Fórmulas

```
# Simulação do impacto de tarifas (ad valorem + específica) no consumo via elasticidade
# Pré-requisitos: `modelo` (OLS com ln_qtd ~ ln_preco) e `df_eua_suco_pd` já carregados

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider, IntSlider, Dropdown, Checkbox

beta_hat = float(modelo.params["ln_preco"])
beta_se = float(modelo.bse["ln_preco"])

def baseline_q(method="last12m"):
    df = df_eua_suco_pd.sort_values(["CO_ANO", "CO_MES"]).copy()
    df["date"] = pd.to_datetime(df["CO_ANO"].astype(str) + "-" + df["CO_MES"].astype(str) + "-01")
    df = df.set_index("date")
    if method == "last12m":
        cutoff = df.index.max() - pd.DateOffset(months=12)
        return float(df.loc[df.index > cutoff, "quantidade_kg"].mean())
    if method == "median":
        return float(df["quantidade_kg"].median())
    return float(df["quantidade_kg"].mean())

def landed_price(p_fob_usdkg, ad_valorem=0.10, specific_usd_per_ton=415.0, apply_adval=True, apply_specific=True):
    s = (specific_usd_per_ton / 1000.0) if apply_specific else 0.0 # USD/kg
    t = ad_valorem if apply_adval else 0.0
    return p_fob_usdkg * (1.0 + t) + s

def simulate_tariff(
    p_fob_usdkg=1.50,
    base_apply_adval=False, base_apply_specific=False, base_adval=0.10, base_specific=415.0,
    scen_apply_adval=True, scen_apply_specific=True, scen_adval=0.10, scen_specific=415.0,
    sims=20000, q0_method="last12m"
):
    q0 = baseline_q(q0_method)

    p_base = landed_price(p_fob_usdkg, base_adval, base_specific, base_apply_adval, base_apply_specific)
    p_scen = landed_price(p_fob_usdkg, scen_adval, scen_specific, scen_apply_adval, scen_apply_specific)

    dp = (p_scen - p_base) / p_base # variação relativa do preço ao importador

    beta_draws = np.random.normal(loc=beta_hat, scale=beta_se, size=sims)
    factor = np.power(1.0 + dp, beta_draws)
    dqpct = (factor - 1.0) * 100.0
    q_new = q0 * factor

    sns.set(style="whitegrid")
    plt.figure(figsize=(9,5))
    sns.histplot(dqpct, bins=50, kde=True, color="#0d6efd", alpha=0.85)
    plt.axvline(np.median(dqpct), color="red", linestyle="--", label=f"Mediana: {np.median(dqpct):.2f}%")
    plt.title(f"ΔQuantidade (%) dado ΔPreço = {dp*100:.2f}% | P_base={p_base:.3f} P_cen={p_scen:.3f} (USD/kg)")
    plt.xlabel("Variação na Quantidade (%)")
    plt.ylabel("Frequência")
    plt.legend()
    plt.tight_layout()
    plt.show()

    print(f"P_FOB (USD/kg): {p_fob_usdkg:.3f}")
    print(f"Preço base importador (USD/kg): {p_base:.3f}")
    print(f"Preço cenário importador (USD/kg): {p_scen:.3f}")
    print(f"ΔPreço: {dp*100:.2f}%")
    print(f"Elasticidade média: {beta_hat:.3f}")
    print(f"ΔQ% Mediana: {np.median(dqpct):.2f}% | P10: {np.percentile(dqpct,10):.2f}% P90: {np.percentile(dqpct,90):.2f}%")
    print(f"Q0 (kg): {q0:,.0f} | Q_new Mediano (kg): {np.percentile(q_new,50):,.0f}")

# Interface:
interact(
    simulate_tariff,
    p_fob_usdkg=FloatSlider(value=1.50, min=0.50, max=4.00, step=0.05, description="FOB (USD/kg)"),
    base_apply_adval=Checkbox(value=False, description="Base: aplica ad valorem"),
    base_apply_specific=Checkbox(value=False, description="Base: aplica específica"),
    base_adval=FloatSlider(value=0.10, min=0.00, max=0.60, step=0.01, description="Base ad val."),
    base_specific=FloatSlider(value=415.0, min=0.0, max=800.0, step=5.0, description="Base específica"),
    scen_apply_adval=Checkbox(value=True, description="Cenário: aplica ad valorem"),
    scen_apply_specific=Checkbox(value=True, description="Cenário: aplica específica"),
    scen_adval=FloatSlider(value=0.10, min=0.00, max=0.60, step=0.01, description="Cenário ad val."),
    scen_specific=FloatSlider(value=415.0, min=0.0, max=800.0, step=5.0, description="Cenário específica"),
    sims=IntSlider(value=20000, min=2000, max=100000, step=2000, description="Simulações"),
    q0_method=Dropdown(options=["last12m", "mean", "median"], value="last12m", description="Base Q0")
)
```

);

FOB (USD/... 1.50

☐ Base: aplica ad valorem

☐ Base: aplica especifica

Base ad val. 0.10

Base espe... 415.00

☒ Cen rio: aplica ad valorem

☒ Cen rio: aplica especifica

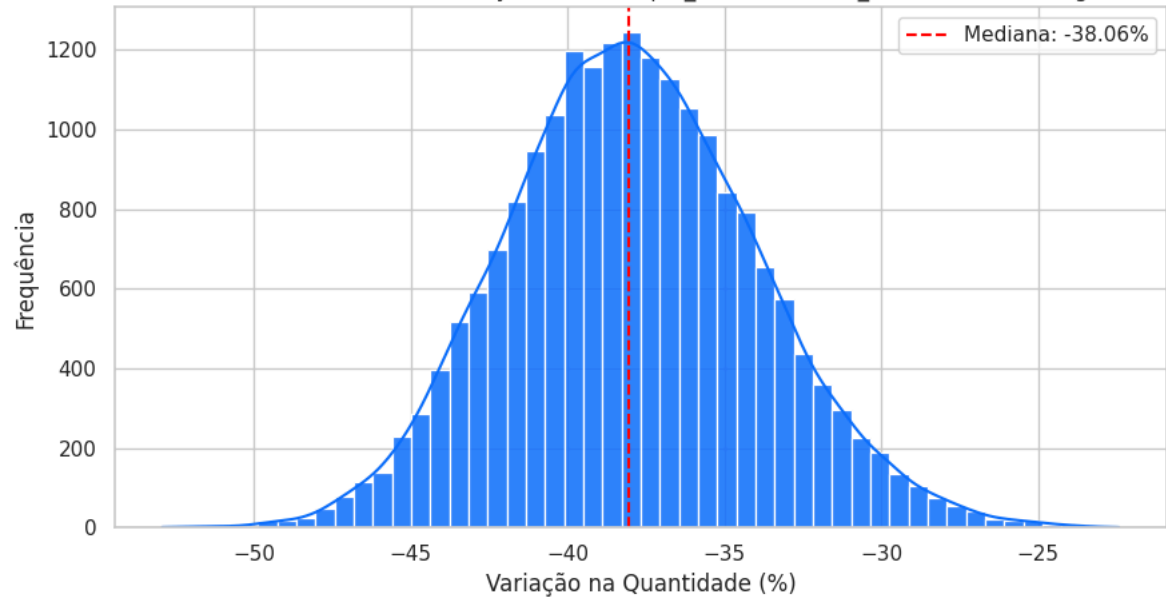
Cen rio ad... 0.10

Cen rio es... 415.00

Simula  es 20000

Base Q  last12m

Δ Quantidade (%) dado Δ Pre o = 37.67% | P_base=1.500 P_cen=2.065 (USD/kg)



P_FOB (USD/kg): 1.500
Pre o base importador (USD/kg): 1.500
Pre o cen rio importador (USD/kg): 2.065
 Δ Pre o: 37.67%
Elasticidade m dia: -1.500
 Δ Q% Mediana: -38.06% | P10: -43.05% P90: -32.80%
Q  (kg): 31,608 | Q_new Mediano (kg): 19,577

Clique duas vezes (ou pressione "Enter") para editar

Interpreta  o da Simula  o — Estrutura Tarif ria Atual

Cen rio

- **Pre o FOB m dio:** US\$ 1,50/kg
- **Tarifas aplicadas:**
 - Ad valorem de 10%
 - Tarifa especifica de US\$ 415 por tonelada (≈ US\$ 0,415/kg)
- **Pre o ao importador (landed price):**
 - Antes das tarifas (base): **US\$ 1,50/kg**
 - Ap s tarifas (cen rio): **US\$ 2,07/kg**
- **Varia  o no pre o: +37,7%**

Resultado da Simula  o (Monte Carlo)

- **Elasticidade-pre o estimada:** -1,50
- **Mediana da varia  o na quantidade:** -38,1%
- **Intervalo prov vel (10%–90%):** de -43,0% a -33,0%

- **Queda estimada no volume exportado:** de ~31,7 mil kg para ~19,6 mil kg (-38%)

Interpretação Econômica

1. Sinal e magnitude esperados

- O aumento de +37,7% no preço ao importador provoca uma queda de cerca de 38% na quantidade demandada — coerente com a elasticidade negativa e elástica ($|\beta| > 1$).
- A resposta é quase proporcional, indicando **alta sensibilidade do mercado norte-americano ao preço**.

2. Efeito econômico

- A **demanda é elástica**, logo o **aumento de preço reduz a receita total** (o ganho unitário é superado pela queda no volume).
- Mesmo sem a sobretaxa de 50%, a tarifa vigente (10% + US\$ 415/t) **aumenta substancialmente o preço efetivo**, comprimindo o consumo.

3. Distribuição de incerteza

- O histograma mostra a probabilidade simulada de diferentes respostas de demanda.
- A distribuição é bem concentrada, sugerindo que o impacto negativo (entre -33% e -43%) é **estatisticamente robusto**.

Conclusão

A estrutura tarifária atual já implica um **encarecimento médio de quase 38% no preço ao importador**, o que reduz as compras dos EUA em torno de **40%**.

Mesmo sem novas sobretaxas, o custo tarifário vigente **já impõe forte perda de competitividade** ao suco brasileiro.

✓ Conclusão Geral — Análise do Comércio Exterior Brasileiro (2010–2025)

A análise integrada do comércio exterior brasileiro, com ênfase na relação com os Estados Unidos e outros parceiros estratégicos, revela um panorama de **forte concentração comercial, sensibilidade de preços e desafios estruturais de diversificação**.

1. Evolução do Comércio Exterior

Entre 2010 e junho de 2025, o Brasil apresentou **crescimento expressivo das exportações**, especialmente após 2021, impulsionado pela valorização de commodities e pela recuperação global pós-pandemia.

As **importações também se expandiram**, mas em ritmo menor, mantendo um **saldo comercial consistentemente positivo** — reflexo do papel dominante dos produtos primários nas exportações.

A sazonalidade e os choques de preços internacionais mostraram-se determinantes nas variações mensais.

2. Estrutura Geográfica

A pauta exportadora brasileira continua **altamente concentrada em poucos mercados**:

- **China e Estados Unidos** respondem juntas por cerca de **60% das exportações totais**.
- A **China** lidera com folga, impulsionada por soja, minério e petróleo.
- Os **Estados Unidos** mantêm papel relevante, sobretudo em produtos industriais, químicos e alimentícios. Essa concentração geográfica evidencia **dependência comercial** e **vulnerabilidade a choques externos**, sejam tarifários, cambiais ou geopolíticos.

3. Estrutura Regional

No plano interno, as exportações concentram-se em **municípios portuários e industriais** — Rio de Janeiro, Parauapebas, São Paulo, Santos e Paranaguá —, evidenciando o peso da infraestrutura logística na inserção internacional do país.

As importações são dominadas por polos industriais e tecnológicos, como **Manaus, São Paulo e Itajaí**, reforçando a centralização nas regiões Sul e Sudeste.